

Local Truth, Global Execution

A Four-Chapter Study of Limits, Presheaves, Sheaves, Toposes, and Cohomology
Through SessionStream

Custom study edition

Revised edition - 16 August 2026

Contents

Preface	1
The four questions	1
How to read this book	2
A recurring SessionStream execution	2
Notation	3
Source discipline	3
Chapter 1 - What Counts as One Coherent State?	4
1.1 The problem before the vocabulary	4
1.1.1 The running consistency record	4
1.2 Minimal category language	5
1.2.1 Category	5
1.2.2 Commutative diagram	6
1.2.3 Isomorphism	7
1.2.4 Monic and epic arrows	7
1.3 Universal properties: specify by comparison	8
1.3.1 A familiar example: pairing	8
1.3.2 Universal property	8
1.3.3 Uniqueness up to unique isomorphism	9
1.4 Products and equalizers: combine, then constrain	9
1.4.1 Product	9
1.4.2 Equalizer	10
1.4.3 Product followed by equalizer	11
1.5 Diagrams, cones, and limits	11
1.5.1 Diagram	11
1.5.2 Cone	12
1.5.3 Limit	12
1.5.4 Familiar limits recovered	12
1.5.5 Finite completeness	13
1.6 Pullbacks: agreement over shared meaning	13
1.6.1 Set-based motivation	13
1.6.2 Definition	13
1.6.3 SessionStream example: snapshot cut and entity set	14
1.6.4 SessionStream example: projection progress	15
1.6.5 SessionStream example: authorization by base change	15
1.6.6 Pulling back a subobject	15
1.6.7 Pullback versus product	16

1.7 Four SessionStream limit studies	16
1.7.1 Study A: a consistent SQLite snapshot	16
1.7.2 Study B: atomic projection progress	16
1.7.3 Study C: snapshot plus live suffix	17
1.7.4 Study D: sufficient API parameters	17
1.8 Exponentials: behavior becomes an object	18
1.8.1 Motivation from sets	18
1.8.2 Definition	18
1.8.3 SessionStream application	19
1.8.4 Why this matters for topoi	19
1.9 Chapter laboratory: specify one transactional witness	19
1.10 Exercises	20
Chapter 2 - What Can Be Known from Each Context?	22
2.1 From one category to another: functors	22
2.1.1 The need for structure-preserving interpretation	22
2.1.2 Definition	23
2.1.3 Example: replay as a functor	23
2.1.4 Example: schema encoding	23
2.1.5 Example: forgetful functor	24
2.1.6 Full, faithful, and essentially surjective	24
2.2 Natural transformations: coherent change across all contexts	24
2.2.1 The problem	24
2.2.2 Definition	25
2.2.3 SessionStream example: migration commutes with replay	25
2.2.4 SessionStream example: store implementations	25
2.2.5 Counterexample: a context-sensitive migration	26
2.2.6 Why naturality is a useful review question	26
2.3 Opposite categories and why restriction reverses arrows	26
2.3.1 Opposite category	26
2.3.2 Contravariant functor	26
2.3.3 Why this matches information restriction	27
2.3.4 SessionStream example: prefix truncation	27
2.3.5 Counterexample: recomputation is not restriction	27
2.4 Choosing the base category of contexts	28
2.4.1 Context	28
2.4.2 Prefix contexts	28
2.4.3 Aspect contexts	28
2.4.4 Product contexts	28
2.4.5 API-information contexts	29
2.4.6 Trace-region contexts	29
2.4.7 Choosing arrows carefully	29
2.5 Presheaves: sets of sections indexed by context	29
2.5.1 Motivation	29
2.5.2 Definition	29
2.5.3 Presheaf laws in software form	30
2.5.4 Example: raw event traces	30
2.5.5 Example: timeline observations	30

2.5.6 Example: invariant presheaf	31
2.5.7 Example: completion presheaf	31
2.5.8 Example: schema knowledge	31
2.6 Natural transformations between presheaves	31
2.6.1 Definition in restriction language	31
2.6.2 SessionStream example: trace summarization	32
2.6.3 SessionStream example: redaction	32
2.7 Fibers, missing coordinates, and parameter sufficiency	32
2.7.1 Fiber	32
2.7.2 Three meanings of “enough”	32
2.7.3 SessionStream example: event identity	33
2.7.4 SessionStream example: deterministic projection	33
2.7.5 Missing coordinate versus failed gluing	33
2.8 Representables and the Yoneda point of view	33
2.8.1 Representable presheaf	33
2.8.2 Yoneda lemma, practical statement	34
2.8.3 SessionStream reading	34
2.8.4 Engineering use	34
2.9 The presheaf category as a software universe	34
2.9.1 Pointwise limits	35
2.9.2 SessionStream application	35
2.10 Chapter laboratory: define a SessionStream presheaf	35
2.11 Exercises	36

Chapter 3 - When Do Compatible Local Views Determine a Whole? 38

3.1 Local pieces are not automatically a whole	38
3.2 Covers: what counts as enough local observation?	39
3.2.1 Open-cover motivation	39
3.2.2 Cover	39
3.2.3 Temporal cover	39
3.2.4 Aspect cover	39
3.2.5 API parameter cover	40
3.3 Pretopologies and sites: lawful coverage policies	40
3.3.1 Pulling back a cover	40
3.3.2 Definition of pretopology	41
3.3.3 Why each axiom matters in software	41
3.3.4 A non-example	41
3.3.5 Grothendieck topology and sieve preview	41
3.4 Matching families and the sheaf condition	41
3.4.1 Compatibility on overlaps	41
3.4.2 Amalgamation	42
3.4.3 Sheaf	42
3.4.4 Separated presheaf	42
3.4.5 Ordinary function example	43
3.4.6 A failing presheaf: bounded summaries	43
3.4.7 A failing presheaf: arbitrary local IDs	43
3.5 The equalizer form of the sheaf condition	43
3.5.1 Two-piece cover	43

3.5.2 General cover	44
3.5.3 Engineering test	44
3.6 Hydration as a gluing theorem	44
3.6.1 The semantic reducer	44
3.6.2 The contexts	45
3.6.3 Sections	45
3.6.4 Matching condition	45
3.6.5 Amalgamation	46
3.6.6 Proposition: snapshot-plus-suffix gluing	46
3.6.7 Mapping to the WebSocket implementation	46
3.6.8 Buffer overflow is an honest no-amalgamation result	47
3.7 Distinguishing local-to-global failures	47
3.7.1 Bad cover	47
3.7.2 Ill-defined restriction	47
3.7.3 Non-matching family	47
3.7.4 No amalgamation	47
3.7.5 Multiple amalgamations	48
3.7.6 Hidden coordinate	48
3.8 Sheafification: repairing a presheaf universally	48
3.8.1 Motivation	48
3.8.2 Definition	48
3.8.3 Software analogies	48
3.8.4 Separated reflection before full sheafification	49
3.9 Toposes: a universe for context-dependent sets	49
3.9.1 Elementary topos	49
3.9.2 Why software engineers should care	49
3.9.3 Subobject	50
3.9.4 Subobject classifier	50
3.9.5 Truth values are contextual	50
3.9.6 Covering sieve	51
3.10 Local truth and Kripke-Joyal reasoning	51
3.10.1 Intuitionistic logic and forcing notation	51
3.10.2 Persistence under refinement	51
3.10.3 Conjunction	51
3.10.4 Disjunction	52
3.10.5 Existential quantification	52
3.10.6 Implication and universal quantification	52
3.10.7 Why excluded middle can fail	52
3.10.8 Failure suspicion as local truth	52
3.11 A topos-oriented architecture review	53
3.12 Chapter laboratory: an executable hydration sheaf test	53
3.13 Exercises	54
Chapter 4 - What Shape Remains When Local Data Does Not Glue?	56
4.1 From an architecture diagram to an overlap complex	56
4.1.1 Graphs are not yet topology	56
4.1.2 Simplicial complex	57
4.1.3 Nerve of a cover	57

4.1.4 SessionStream architecture nerve	57
4.1.5 A face means more than pairwise tests	58
4.2 The Cech nerve and repeated overlaps	58
4.2.1 Definition	58
4.2.2 SessionStream reading	58
4.3 Why cohomology needs coefficients	59
4.3.1 Additive comparison data	59
4.3.2 Constant coefficients	59
4.3.3 Sheaf coefficients	60
4.4 Cochains and coboundaries	60
4.4.1 Cochains	60
4.4.2 Engineering interpretation	60
4.4.3 Coboundary from vertices to edges	60
4.4.4 Coboundary from edges to faces	61
4.4.5 Why boundary of boundary is zero	61
4.5 Cohomology	61
4.5.1 Meaning of H^0	62
4.5.2 Meaning of H^1	62
4.5.3 Closed versus exact	62
4.6 Worked calculation: a four-component cursor loop	62
4.6.1 Vertex coordinates	63
4.6.2 Edge offsets	63
4.6.3 Consistent calibration	63
4.6.4 Engineering interpretation	64
4.6.5 What the calculation does not prove	64
4.7 Filled triangles and higher-order witnesses	64
4.7.1 Transaction as a face	65
4.7.2 Pairwise atomicity is insufficient	65
4.8 Cellular sheaves: different data at different cells	65
4.8.1 A simple linear sheaf model	65
4.8.2 SessionStream example	66
4.8.3 Interpreting H^1	66
4.9 An obstruction catalogue for SessionStream	66
4.9.1 Cursor holonomy: a good cohomology candidate	66
4.9.2 Snapshot race: usually a missing joint witness first	66
4.9.3 Stable retry identity: usually a missing coordinate	66
4.9.4 Deterministic replay: usually functoriality	67
4.9.5 Hydration gap: usually cover failure	67
4.9.6 Conflicting migrations: possible cocycle problem	67
4.9.7 Observer evidence loss: epistemic limitation	67
4.10 Computing cohomology from matrices	67
4.10.1 Workflow	68
4.10.2 Graph cycle rank	68
4.10.3 Numerical caution	68
4.11 Build ss-sheafcheck: an executable research tool	68
4.11.1 Proposed package layout	68
4.11.2 Model file	69

4.11.3 Validation phases	69
4.11.4 Diagnostic output	70
4.11.5 Trace adapter	70
4.11.6 Capstone research questions	71
4.12 A final diagnostic order	71
4.13 Exercises	71
Closing Perspective	74
Appendix A - Definition and Dependency Map	75
A.1 Arrows and universal constructions	75
A.2 Functors and presheaves	76
A.3 Sheaves, sites, and truth	77
A.4 Nerves and cohomology	78
A.5 Failure-to-concept index	79
Appendix B - Selected Hints and Solutions	80
B.1 Chapter 1	80
Exercise 1.6: uniqueness of products	80
Exercise 1.10: pullback in sets	80
Exercise 1.14: constant on fibers	81
Exercise 1.19: pullback from product and equalizer	81
B.2 Chapter 2	81
Exercise 2.3: hidden randomness	81
Exercise 2.6: pointwise migration without naturality	81
Exercise 2.11: composition-law mutation	81
Exercise 2.17: representable on a three-object chain	82
B.3 Chapter 3	82
Exercise 3.6: equalizer form	82
Exercise 3.7: separated but not a sheaf	82
Exercise 3.10: hydration induction	82
Exercise 3.12: late buffer	82
Exercise 3.18: sieves on a three-object chain	83
B.4 Chapter 4	83
Exercise 4.8: boundary of boundary	83
Exercise 4.10: zero circulation implies exactness	83
Exercise 4.14: square with a diagonal and one filled triangle	84
Exercise 4.17: multiple global sections	84
Exercise 4.29: zero H^1 is limited evidence	84
Appendix C - Source Map and Parallel Study Route	85
C.1 Goldblatt route	85
C.2 SessionStream source map	85
C.3 Twelve-week route	86
C.4 Study notebook template	87
C.5 Completion criteria	87

Preface

This is a custom textbook for a software developer learning category theory, presheaves, sheaves, topoi, and cohomology through one concrete system: **SessionStream**.

SessionStream is a Go framework for session-scoped, event-driven applications. A typed command enters a hub; a handler publishes canonical backend events; UI and timeline projections interpret those events; durable timeline entities support hydration; and reconnecting WebSocket clients receive a snapshot before later live events. The system therefore contains many partial, overlapping descriptions of one execution. That is exactly the kind of situation in which local-to-global mathematics becomes useful.

The pedagogical model is Robert Goldblatt’s *Topoi: The Categorical Analysis of Logic*. Goldblatt commonly begins with a familiar construction in sets, isolates the arrow pattern, states an abstract definition, works several examples, proves a structural fact, points out a failed converse or counterexample, and then assigns exercises. This book follows that rhythm. It does **not** reproduce Goldblatt’s prose or attempt to replace his book. The definitions and proofs are independently written and the examples are reorganized around SessionStream.

The source study is frozen as follows:

- Goldblatt’s book as supplied with this project, especially Chapters 3, 4, 9, and 14;
- the SessionStream Architecture Garden note as modified on 15 August 2026;
- the SessionStream repository at commit d62dca9f5efa2e3094d6c62e5ead5ed0c88fd35c, merged on 13 August 2026.

The implementation will change. Treat this as a mathematical study of a named snapshot, not as permanent API documentation.

The four questions

The book has four large chapters, each governed by one question.

1. **What counts as one coherent state?** Categories, universal properties, limits, pullbacks, and exponentials.
2. **What can be known from each context?** Functors, natural transformations, opposite categories, presheaves, restrictions, and parameter sufficiency.
3. **When do compatible local views determine a whole?** Covers, sites, sheaves, sheafification, topoi, and local truth.
4. **What shape remains when they do not?** Nerves, cochains, cohomology, obstruction diagnostics, and an executable SessionStream research tool.

These are not four unrelated subjects. They form one chain:

```
joint witnesses
  -> context-indexed information
    -> local compatibility and gluing
      -> higher-order shape of failure
```

How to read this book

Every important term is introduced in six passes.

1. **Need.** What problem forces us to name this idea?
2. **Set example.** What does it mean for ordinary sets and functions?
3. **Definition.** What is the arrow-only or context-only form?
4. **SessionStream example.** Where does it appear in the running system?
5. **Failure mode.** What goes wrong when the conditions are absent?
6. **Exercise.** Can you use the definition without the prose beside you?

When a definition still feels formal, ask three questions:

- What are the candidate objects or states?
- What information must a comparison map preserve?
- What universal, restriction, or gluing property singles out the desired object?

Do not rush to cohomology. The most useful concepts in this book are often the earlier ones: a missing coordinate, a bad restriction map, an invalid cover, or a non-atomic witness explains many software failures before higher algebra is needed.

A recurring SessionStream execution

For a fixed session s , imagine the following accepted backend events:

```
ordinal 1: UserMessageAccepted(message = u1)
ordinal 2: InferenceStarted(message = a1)
ordinal 3: TokensDelta(message = a1, chunk = "cat")
ordinal 4: TokensDelta(message = a1, chunk = "egory")
ordinal 5: InferenceFinished(message = a1)
```

From the same history we obtain several views:

- the canonical event prefix through ordinal 5;
- the UI batches sent at each ordinal;
- the durable timeline entity for $u1$;
- the durable timeline entity for $a1$;
- a snapshot whose `SnapshotOrdinal` is 5;
- a projection cursor claiming that the timeline projector reached 5;
- a browser state reconstructed from a snapshot and later live events;
- an observer trace recording transport stages.

These values are related, but they are not aliases. An event is not an entity. An entity is not a snapshot. A snapshot cut is not automatically an event-store cursor. A projection cursor

is a claim about interpreter progress, not merely an integer. Much of this book consists of making those relationships explicit.

Notation

- Categories are written $\mathcal{C}, \mathcal{D}, \mathcal{E}$.
- Objects are A, B, C ; arrows are $f : A \rightarrow B$.
- Composition is $g \circ f$: first f , then g .
- $1_A : A \rightarrow A$ is the identity arrow.
- $\mathcal{C}^{op}$ is the opposite category.
- A presheaf is a functor $F : \mathcal{C}^{op} \rightarrow \mathbf{Set}$.
- A section over a context U is an element $s \in F(U)$.
- A restriction from U to V is written $s|_V$ when V is the poorer context.
- $E_{\leq n}$ is the canonical event prefix through ordinal n .
- T_n is timeline materialization claimed through n .
- S_n is a snapshot at cut n .
- $L_{(n,m]}$ is the live suffix after n through m .
- H^k is used only after an additive coefficient system has been specified.

Source discipline

Statements about the supplied Goldblatt text are confined to its pedagogical route and terminology. Statements about SessionStream are tied to the frozen source snapshot. The categorical and sheaf-theoretic models are proposed interpretations. They expose useful proof obligations, but they do not by themselves prove production correctness.

Chapter 1 - What Counts as One Coherent State?

1.1 The problem before the vocabulary

Suppose a reconnecting client is told all of the following:

```
snapshot ordinal:           42
maximum entity last ordinal: 42
projection cursor:          42
first live batch ordinal:    43
```

These numbers fit together. Now change the second line:

```
maximum entity last ordinal: 43
```

The client has received an entity that claims knowledge of an event beyond the advertised snapshot cut. Each field is individually well-typed. The failure lies in their **joint relationship**.

This chapter develops the mathematical language for such questions. The central idea is not that one component owns the truth. It is that a coherent global state is a **joint witness** satisfying several projections and compatibility equations at once.

Goldblatt reaches the same general idea by observing that products, equalizers, and terminal objects share one pattern: other candidates factor uniquely through a canonical candidate. That pattern is a **universal property**. Limits package many such joint-witness problems into one definition.

1.1.1 The running consistency record

We will repeatedly use a simplified diagnostic record:

```
type CutRecord struct {
    EventCursor      uint64
    ProjectionCursor  uint64
    SnapshotOrdinal   uint64
    MaxEntityOrdinal  uint64
    FirstLiveOrdinal  uint64
}
```

A locally valid value merely satisfies field-level typing. A globally valid value might satisfy laws such as

$$\text{MaxEntityOrdinal} \leq \text{SnapshotOrdinal},$$

$$\text{ProjectionCursor} \leq \text{EventCursor},$$

$$\text{FirstLiveOrdinal} > \text{SnapshotOrdinal}.$$

The set of records satisfying all the laws is not obtained by looking at any one field. It is carved out by relationships among fields. Equalizers and pullbacks will give us two ways to describe such lawful subsets.

Reading checkpoint. Before learning any definition, say what the desired mathematical object must do: it must collect several observations, preserve their individual values, and reject combinations that disagree on shared meaning.

1.2 Minimal category language

The user of a library rarely cares how a value is represented internally. They care which operations are available and how operations compose. Category theory takes that operational point of view seriously.

1.2.1 Category

Need. We need a language that speaks about transformations without reducing them to their memory representation.

Set example. In the category **Set**, objects are sets and arrows are functions. Functions compose, each set has an identity function, composition is associative, and identities do nothing.

For an arrow $f : A \rightarrow B$, A is its **domain** and B its **codomain**. Two arrows are **composable** when the codomain of the first is the domain of the second. The words **arrow** and **morphism** will be used interchangeably.

Definition 1.1 (Category). A category $\mathcal{C}$ consists of:

1. a collection of objects;
2. for each ordered pair A, B , a collection of arrows $f : A \rightarrow B$;
3. for composable arrows $A \xrightarrow{f} B \xrightarrow{g} C$, a composite $g \circ f : A \rightarrow C$;
4. for every object A , an identity arrow $1_A : A \rightarrow A$;

such that composition is associative and identities are left and right units:

$$h \circ (g \circ f) = (h \circ g) \circ f,$$

$$1_B \circ f = f = f \circ 1_A.$$

SessionStream example. Let $\mathcal{P}_s$ be a category whose objects are event-prefix cuts for one session:

$$0, 1, 2, \dots$$

There is one arrow $m \rightarrow n$ precisely when $m \leq n$. The arrow means “extend the observed history from cut m to cut n .” Composition is transitivity of extension. The identity $n \rightarrow n$ means no extension.

This is a **poset category**: a partially ordered set regarded as a category with at most one arrow between any two objects.

Second example. Let objects be representations of a backend event: Go protobuf value, protobuf bytes, protobuf JSON, browser object. Arrows are chosen codecs. This forms a category only if the chosen transformations compose associatively and identities behave as identities. A lossy JSON conversion may still be an arrow, but a claimed round-trip isomorphism would fail.

Failure mode. A collection of functions is not automatically a useful category. If an operation advertised as composition depends on hidden mutable state, then the associative law may fail observationally.

1.2.2 Commutative diagram

Need. Two implementation routes may start and end at the same types. We need to say whether they have the same observable effect.

Definition 1.2 (Commutative diagram). A diagram commutes when any two directed paths with the same start and end have equal composites.

For a square

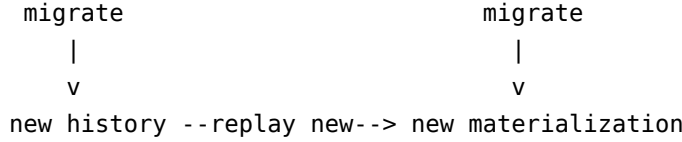
$$\begin{array}{ccc} A & \xrightarrow{f} & B \\ | & & | \\ g & & h \\ | & & | \\ v & & v \\ C & \xrightarrow{k} & D \end{array}$$

commutativity means

$$h \circ f = k \circ g.$$

SessionStream example. Let $E_{\leq n}$ be an event prefix, R a replay operation, P a timeline projection, and M a migration from schema version 1 to version 2. A desirable square is

$$\begin{array}{ccc} \text{old history} & \xrightarrow{\text{replay}} & \text{old materialization} \\ | & & | \end{array}$$



The square commutes when migrating before replay gives the same result as replaying before migrating.

Failure mode. A migration that patches stored rows but does not transform replayed events may make the square fail. The diagram is then not decoration; it is a testable route-independence claim.

1.2.3 Isomorphism

Need. Mathematical constructions are usually determined by behavior, not by one concrete struct layout.

Definition 1.3 (Isomorphism). An arrow $f : A \rightarrow B$ is an isomorphism if there is an arrow $g : B \rightarrow A$ such that

$$g \circ f = 1_A, \quad f \circ g = 1_B.$$

Then A and B are isomorphic, written $A \cong B$.

Set example. An isomorphism in **Set** is a bijection.

SessionStream example. A snapshot represented as a sorted slice of unique entities may be isomorphic, for the intended API, to a map keyed by (Kind, Id) if conversion in both directions preserves exactly the observable data and ordering is declared irrelevant. If stable presentation order is observable, the two representations are not isomorphic in that richer category.

Important lesson. “Isomorphic” always depends on the category. Two values can be isomorphic as raw sets and non-isomorphic as ordered event histories.

1.2.4 Monic and epic arrows

These notions are less central than limits but useful for later subobjects.

Definition 1.4 (Monic). An arrow $m : A \rightarrow B$ is monic if

$$m \circ f = m \circ g \implies f = g$$

for all $f, g : X \rightarrow A$.

It is left-cancellable. In **Set**, monic functions are injective.

Definition 1.5 (Epic). An arrow $e : A \rightarrow B$ is epic if

$$f \circ e = g \circ e \implies f = g$$

for all $f, g : B \rightarrow Y$.

It is right-cancellable. In **Set**, epic functions are surjective.

SessionStream example. Including the lawful snapshot records into all field-typed records is monic: two lawful records that become equal after inclusion were already equal. This will later let us regard a law as a **subobject**.

Student checkpoint. A category tells you what transformations count. A commutative diagram states route independence. An isomorphism states reversible equivalence in the chosen notion of structure. Monic and epic are cancellation properties, not synonyms for injective and surjective outside **Set**.

1.3 Universal properties: specify by comparison

Software specifications often say how to construct a value. A universal property says how the value compares with every other candidate.

1.3.1 A familiar example: pairing

Given sets A and B , the Cartesian product $A \times B$ comes with projections

$$\pi_A : A \times B \rightarrow A, \quad \pi_B : A \times B \rightarrow B.$$

Given any set X and functions $f : X \rightarrow A, g : X \rightarrow B$, there is exactly one function

$$\langle f, g \rangle : X \rightarrow A \times B$$

whose projections recover f and g :

$$\pi_A \circ \langle f, g \rangle = f, \quad \pi_B \circ \langle f, g \rangle = g.$$

The formula $x \mapsto (f(x), g(x))$ proves existence in **Set**. The equations prove that the product has the right abstract behavior.

1.3.2 Universal property

Need. We want a representation-independent specification of a “best” candidate.

Definition 1.6 (Universal property, working form). A candidate object is universal for a class of structured candidates when every other candidate admits a unique structure-preserving arrow to it, or uniquely from it, according to the construction.

A **factorization** of an arrow $h : X \rightarrow A$ through $i : E \rightarrow A$ is an arrow $k : X \rightarrow E$ satisfying $i \circ k = h$. Thus a universal property does not merely assert that a map exists; it says every candidate map factors in one and only one prescribed way.

For limits, the arrows go **from other candidates into the universal candidate**. For colimits, the direction is reversed. This book concentrates on limits.

A universal property has three parts:

1. **candidate data** - the object and its structure arrows;
2. **existence** - every competing candidate factors through it;
3. **uniqueness** - there is only one such factorization.

Why uniqueness matters. Existence says the universal object is sufficient. Uniqueness says it introduces no arbitrary hidden choice.

1.3.3 Uniqueness up to unique isomorphism

A universal object need not be literally the same struct or set as another universal object. It is determined up to a uniquely determined isomorphism that preserves the structure arrows.

Proposition 1.7. Any two products of A and B are uniquely isomorphic in a way that commutes with their projections.

Proof sketch. Let (P, p_A, p_B) and (Q, q_A, q_B) both satisfy the product property. Universality of P gives a unique arrow $u : Q \rightarrow P$ compatible with projections. Universality of Q gives a unique arrow $v : P \rightarrow Q$. Both $u \circ v$ and 1_P have the same composites with p_A, p_B , so uniqueness forces $u \circ v = 1_P$. Similarly $v \circ u = 1_Q$. The same uniqueness argument shows there is no other projection-preserving isomorphism. $\square$

Engineering application. An in-memory hydration store and a SQLite hydration store should not be expected to have the same implementation. A universal specification can instead require that both present the same observable limit-like object to clients.

1.4 Products and equalizers: combine, then constrain

Products and equalizers are the two simplest ingredients for many finite limits.

1.4.1 Product

Definition 1.8 (Product). A product of objects A and B in $\mathcal{C}$ is an object $A \times B$ with arrows

$$\pi_A : A \times B \rightarrow A, \quad \pi_B : A \times B \rightarrow B$$

such that for any $f : X \rightarrow A$ and $g : X \rightarrow B$, there is a unique $\langle f, g \rangle : X \rightarrow A \times B$ satisfying the projection equations.

SessionStream example 1: combined observation. Let A be possible snapshot-cut observations and B possible entity-set observations. Their product contains arbitrary pairs

$$(n, \mathcal{E}).$$

At this stage no consistency relation is imposed. Product means “observe both,” not “they agree.”

SessionStream example 2: product projection. Given one canonical event, run an independent UI projector and audit projector:

$$(P_{UI} \times P_A)(e, s) = (P_{UI}(e, s), P_A(e, s)).$$

The projections recover each interpretation. This product is lawful only when the two projectors do not secretly mutate shared input.

Counterexample. A struct with fields `SnapshotOrdinal` and `Entities` is not automatically a categorical product. The API may enforce additional invariants, omit possible pairs, or make field access effectful. Product is a universal property, not a syntax shape.

1.4.2 Equalizer

Need. A product collects independent observations. We now need to keep only values on which two interpretations agree.

Set example. For functions $f, g : A \rightarrow B$, define

$$E = \{a \in A \mid f(a) = g(a)\}.$$

The inclusion $i : E \hookrightarrow A$ equalizes f and g because $f \circ i = g \circ i$.

Definition 1.9 (Equalizer). An equalizer of parallel arrows $f, g : A \rightarrow B$ is an arrow $i : E \rightarrow A$ such that

$$f \circ i = g \circ i,$$

and whenever $h : X \rightarrow A$ also satisfies $f \circ h = g \circ h$, there is a unique $k : X \rightarrow E$ with

$$i \circ k = h.$$

SessionStream example 1: sound snapshots. Let A be all pairs $(n, \mathcal{E})$ of a claimed cut and entity set. Define two maps into a set of truth values:

$$f(n, \mathcal{E}) = \text{true},$$

$$g(n, \mathcal{E}) = [\forall x \in \mathcal{E}, \text{last}(x) \leq n].$$

The equalizer is the subset of pairs satisfying snapshot soundness.

A less artificial formulation maps to a set of violation reports. Let

$$\text{viol}(n, \mathcal{E}) = \{x \in \mathcal{E} \mid \text{last}(x) > n\},$$

and let 0 be the constant empty report. The equalizer of `viol` and 0 is precisely the lawful subset.

SessionStream example 2: deterministic replay. Let A be replay inputs. Let

$$f : A \rightarrow T$$

run the live projection path and

$$g : A \rightarrow T$$

run the rebuild path. Their equalizer is the set of inputs on which replay agrees with live processing.

Proposition 1.10. Every equalizer arrow is monic.

Proof. If $i : E \rightarrow A$ equalizes f, g and $i \circ u = i \circ v$, then the common composite is an arrow into A equalizing f, g . By uniqueness of factorization through i , $u = v$. $\square$

Failed converse. A monic need not be an equalizer in every category. Do not replace a universal property with a familiar set-theoretic characterization without checking the category.

1.4.3 Product followed by equalizer

Many consistency objects have this pattern:

1. take a product of all local observations;
2. use an equalizer to keep only tuples whose overlap views agree.

For two views $F(U)$ and $F(V)$ that both restrict to $F(U \cap V)$, compatible pairs form

$$\text{Eq}(F(U) \times F(V) \rightrightarrows F(U \cap V)).$$

This formula will reappear as the two-set sheaf condition in Chapter 3.

1.5 Diagrams, cones, and limits

Products and equalizers look different until we focus on their arrows.

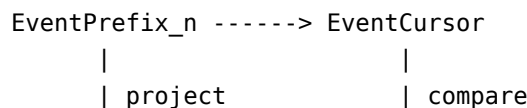
1.5.1 Diagram

Need. A coherent state often depends on more than two objects and more than one compatibility equation. We need a finite schema of objects and arrows.

Definition 1.11 (Diagram). A diagram D in a category $\mathcal{C}$ is a collection of objects and arrows in $\mathcal{C}$ arranged according to some indexing shape.

Formally, a diagram of shape J is a functor $D : J \rightarrow \mathcal{C}$. The informal picture is enough at first: boxes are objects, arrows are required relationships.

SessionStream example. Consider the diagram



$\vee$
 $\vee$
TimelineState_n ----> ProjectionCursor

The actual arrows must be chosen precisely: perhaps each state maps to the greatest ordinal it represents. The diagram does not become meaningful until those maps are defined.

1.5.2 Cone

Need. We want one candidate object that gives a compatible observation at every object of the diagram.

Definition 1.12 (Cone). A cone from an object X to a diagram D consists of an arrow

$$\lambda_j : X \rightarrow D(j)$$

for every object j in the shape, such that every triangle induced by an arrow of the diagram commutes.

The object X is the cone's **apex**. The arrows λ_j are its **legs**.

SessionStream reading. A cone is a proposed global execution witness together with all the observations it induces: event prefix, materialized state, cursor values, snapshot, and client state. Commutativity says those observations tell a consistent story.

A tuple of values is not enough. The cone must satisfy every compatibility arrow in the diagram.

1.5.3 Limit

Definition 1.13 (Limit). A limit of a diagram D is a cone (L, λ_j) such that every other cone (X, μ_j) factors through it by a unique arrow $u : X \rightarrow L$:

$$\lambda_j \circ u = \mu_j$$

for every j .

The limit is the universal compatible cone.

Interpretation. The limit contains exactly the information needed to specify a compatible family of observations, with no extra arbitrary choice.

1.5.4 Familiar limits recovered

Definition (Terminal object). A terminal object 1 is an object such that every object X has exactly one arrow $X \rightarrow 1$. A one-element set is terminal in **Set**. In the bounded prefix category $0 \rightarrow 1 \rightarrow \dots \rightarrow N$, the final cut N is terminal.

- The limit of the empty diagram is a terminal object.
- The limit of two disconnected objects is their **product**.
- The limit of two parallel arrows is their **equalizer**.
- The limit of a cospan $A \rightarrow C \leftarrow B$ is a **pullback**.

This is the conceptual step at the limits section of Goldblatt Chapter 3: several constructions are instances of one universal pattern.

1.5.5 Finite completeness

Definition 1.14 (Finite limit). A finite limit is a limit of a finite diagram.

Definition 1.15 (Finitely complete category). A category is finitely complete if every finite diagram has a limit.

A standard result says it is enough to have a terminal object and pullbacks; equivalently, a terminal object, binary products, and equalizers. This matters because topos theory assumes finite-limit structure: internal predicates, substitutions, and local compatibility all depend on stable finite joint witnesses.

Student checkpoint. A diagram is a constraint schema. A cone is one candidate global witness. A limit is the universal candidate. A finite-limit category can build coherent finite combinations throughout the theory.

1.6 Pullbacks: agreement over shared meaning

The pullback is the most important limit for this book.

1.6.1 Set-based motivation

Suppose

$$f : A \rightarrow C, \quad g : B \rightarrow C.$$

The pullback set is

$$A \times_C B = \{(a, b) \in A \times B \mid f(a) = g(b)\}.$$

It pairs an A -value and a B -value only when their images in the common comparison space C agree.

This is a typed join:

```
SELECT *  
FROM A JOIN B  
ON f(A) = g(B)
```

but with a universal property, not merely a query syntax.

1.6.2 Definition

Definition 1.16 (Pullback). Given arrows $f : A \rightarrow C$ and $g : B \rightarrow C$, a pullback consists of an object P and arrows

$$p_A : P \rightarrow A, \quad p_B : P \rightarrow B$$

such that

$$f \circ p_A = g \circ p_B,$$

and for every X with arrows $x_A : X \rightarrow A, x_B : X \rightarrow B$ satisfying

$$f \circ x_A = g \circ x_B,$$

there is a unique $u : X \rightarrow P$ with

$$p_A \circ u = x_A, \quad p_B \circ u = x_B.$$

The commutative square

$$\begin{array}{ccc} P & \xrightarrow{p_B} & B \\ | & & | \\ p_A & & g \\ | & & | \\ A & \xrightarrow{f} & C \end{array}$$

is a **pullback square** or **Cartesian square**.

1.6.3 SessionStream example: snapshot cut and entity set

Let

- A be snapshot metadata records;
- B be entity collections;
- C be ordinal claims.

Define

$$f : A \rightarrow C$$

by returning the snapshot ordinal, and

$$g : B \rightarrow C$$

by returning the maximum event ordinal represented by the entity collection.

The literal pullback requires equality:

$$\text{snapshotOrdinal}(a) = \text{maxEntityOrdinal}(b).$$

That may be too strong because a valid snapshot can contain entities last changed before the cut. To model the actual inequality, change the comparison object. Let C be pairs (m, n)

with $m \leq n$, and map the snapshot to its upper bound and the entity set to its represented maximum. The modeling lesson is important:

A pullback enforces equality in the comparison object. To represent a richer relation, encode that relation in the comparison object or use a subobject/predicate.

1.6.4 SessionStream example: projection progress

Let

- M be materialization states;
- K be projection-cursor states;
- O be ordinal claims.

Map $M \rightarrow O$ to the greatest event actually represented and $K \rightarrow O$ to the cursor's claimed progress. The pullback consists of pairs in which represented and claimed progress agree.

The current implementation performs entity application and projection-cursor advancement as separate calls. If one succeeds and the other fails, the persisted state can lie outside the intended pullback. A transaction can be viewed as constructing one stronger witness in which both facts become visible together.

1.6.5 SessionStream example: authorization by base change

Suppose a client connection maps to an authenticated principal and a session maps to its owner or policy context. A subscription request should exist only when the two map to an allowed relationship. The authorized-request object can often be modeled as a pullback of principal evidence and session policy evidence over an authorization relation.

This does not make authorization “category theory.” It clarifies the joint witness: connection identity alone is insufficient; session identity alone is insufficient; an authorized request is a compatible pair over a policy decision.

1.6.6 Pulling back a subobject

Suppose $m : S \hookrightarrow C$ is a subobject representing lawful values and $f : A \rightarrow C$ is an observation. Pulling m back along f gives the subobject of A whose observations are lawful.

In sets, this is simply the inverse image

$$f^{-1}(S) = \{a \in A \mid f(a) \in S\}.$$

SessionStream example. Let $S \subseteq \mathbb{N} \times \mathbb{N}$ contain pairs (*entityOrdinal*, *snapshotOrdinal*) with the first no greater than the second. Pulling this predicate back along a function that extracts those ordinals from snapshots yields the set of sound snapshots.

This stability under change of context will later become essential for sites and local truth.

1.6.7 Pullback versus product

A product answers:

What if I observe both values independently?

A pullback answers:

What if I observe both values and require them to agree after translating them into shared meaning?

A struct is often product-shaped. An invariant turns it into a pullback- or equalizer-shaped subobject.

1.7 Four SessionStream limit studies

1.7.1 Study A: a consistent SQLite snapshot

The SQLite snapshot path conceptually reads two things:

1. the cut n ;
2. entity rows $\mathcal{E}$.

If they are read in different database moments, there may be no actual database state whose projections are both observations. The pair is locally plausible but lacks a global cone.

A read transaction changes the semantics. The database snapshot becomes an apex from which both the cursor and row set are projected. The transaction is not merely faster or slower; it supplies the missing joint witness.

Law. Every entity returned in a snapshot must satisfy

$$\text{LastEventOrdinal}(x) \leq \text{SnapshotOrdinal}.$$

Test design. Run concurrent Apply and Snapshot operations under a scheduler that tries to interleave cursor and row reads. Assert that no returned record violates the law. A deliberately split implementation should produce a counterexample under a strong enough harness.

1.7.2 Study B: atomic projection progress

The hub's processing route contains distinct steps:

```
append canonical event
read current timeline view
run projections
apply timeline entities
advance projection cursor
fan out UI events
```


Different failure policies may be appropriate for each step. The durable claim “timeline projector has processed through n ” should, however, correspond to materialization through n .

A suitable invariant object is the pullback of

$$\text{represented} : M \rightarrow O$$

and

$$\text{claimed} : K \rightarrow O.$$

Possible repairs include:

- one database transaction for materialization and cursor advance;
- a write-ahead state machine representing incomplete progress explicitly;
- an idempotent replay protocol that never treats the cursor as stronger evidence than it is.

The categorical model does not choose the implementation. It identifies the joint claim that must be witnessed.

1.7.3 Study C: snapshot plus live suffix

Let S_n be a snapshot representing a prefix through n , and let $L_{(n,m]}$ be live outputs attributable to events after n through m .

A client reconstruction is a joint witness over their shared boundary. The key equations are:

$$\text{cut}(S_n) = n,$$

$$\min \text{ord}(L_{(n,m]}) > n,$$

$$\text{reduce}(S_n, L_{(n,m]}) = C_m.$$

Chapter 3 will reinterpret this as sheaf gluing. Here the limit lesson is enough: the client state is not arbitrary concatenation; it is a compatible cone over snapshot and suffix observations.

1.7.4 Study D: sufficient API parameters

Let X be the space of complete semantic transaction states and P the values exposed by an endpoint. The **image** of a function is the set of outputs it actually attains. Restriction gives

$$r : X \rightarrow P.$$

For a supplied request $p \in P$, the fiber

$$r^{-1}(p)$$

is the set of global completions consistent with the parameters.

- Empty fiber: the request is impossible.
- Singleton fiber: the parameters determine the entire semantic state.
- Multiple-element fiber: the request is underdetermined.

For an invariant $I : X \rightarrow \{0, 1\}$, the parameters are sufficient to decide I when I is constant on the fiber:

$$r(x) = r(y) \implies I(x) = I(y).$$

Equivalently, I factors through r : there is a function $\bar{I} : P \rightarrow \{0, 1\}$ with

$$I = \bar{I} \circ r.$$

This factorization criterion is often more direct than cohomology for API sufficiency.

1.8 Exponentials: behavior becomes an object

Topos theory requires more than finite limits. It also needs function-like objects.

1.8.1 Motivation from sets

For sets A and B , let B^A be the set of functions $A \rightarrow B$. There is an evaluation function

$$\text{ev} : B^A \times A \rightarrow B, \quad \text{ev}(f, a) = f(a).$$

Given any $g : C \times A \rightarrow B$, fixing $c \in C$ produces a function $A \rightarrow B$. Therefore g corresponds to a unique function

$$\bar{g} : C \rightarrow B^A.$$

This is currying.

1.8.2 Definition

Definition 1.17 (Exponential). In a category with products, an exponential B^A is an object with an evaluation arrow

$$\text{ev} : B^A \times A \rightarrow B$$

such that every $g : C \times A \rightarrow B$ factors uniquely as

$$g = \text{ev} \circ (\bar{g} \times 1_A)$$

for a unique $\bar{g} : C \rightarrow B^A$.

Equivalently, for every object C there is a bijection, coherent as C varies (Chapter 2 will call this coherence *naturality*),

$$\mathcal{C}(C \times A, B) \cong \mathcal{C}(C, B^A).$$

Definition 1.18 (Cartesian closed). A category is Cartesian closed if it has finite products and exponentials for every pair of objects.

1.8.3 SessionStream application

A projection type resembles a function object only after its dependencies are explicit. Consider

```
Project(event, session, timelineView) -> []TimelineEntity
```

If the projector also reads a clock, random generator, network, mutable global, or drifting schema, then its true domain is larger than the declared arguments. Treating it as a point of an exponential based on the smaller domain is false modeling.

Exponentials therefore motivate a dependency question:

What object contains the complete input context required to make this behavior a value?

A deterministic projector can be treated as a global element of a function object. A projector with hidden effects belongs in a richer category, perhaps one of effectful computations, not in the pure function space being claimed.

1.8.4 Why this matters for topoi

An elementary topos is, in one concise definition, a Cartesian closed category with a **sub-object classifier**: an object that turns suitably represented predicates into characteristic arrows. Chapter 3 gives the precise universal property. Finite limits give joint coherent state. Exponentials make behavior internal. The classifier makes predicates and contextual truth internal.

1.9 Chapter laboratory: specify one transactional witness

Choose one SessionStream law:

- consistent snapshot cut;
- atomic timeline progress;
- deterministic replay;
- snapshot-plus-suffix completeness;
- stable retry identity.

Perform the following steps.

1. List the observations involved.
2. Define a comparison object capturing their shared meaning.
3. Draw the product of independent observations.
4. State the equations or predicate that cuts out lawful combinations.
5. Decide whether an equalizer, pullback, or general finite limit best expresses the law.
6. Name the apex that an implementation could provide: transaction, event identity, state machine, replay witness, or typed parameter object.
7. Write one property test that tries to falsify the universal claim.

A good result is not a diagram alone. It includes the exact maps.

1.10 Exercises

The exercises are ordered roughly from direct use of definitions to architecture design. A star marks exercises with a selected solution or hint in Appendix B.

Exercise 1.1. Show that the natural numbers with one arrow $m \rightarrow n$ exactly when $m \leq n$ form a category.

Exercise 1.2. Give two different categories whose objects include `SessionStream` events but whose isomorphisms differ. Explain which structure each category preserves.

Exercise 1.3. Draw a commutative square expressing that protobuf binary encode/decode and protobuf JSON encode/decode preserve one chosen event value. State exactly what equality means at the bottom-right object.

Exercise 1.4. Prove that isomorphisms are monic and epic.

Exercise 1.5. Give an example of a Go struct with two fields that is not a categorical product for the intended API.

Exercise 1.6. Prove Proposition 1.7 in full detail, including uniqueness of the structure-preserving isomorphism.

Exercise 1.7. For functions $f, g : A \rightarrow B$, verify directly that the inclusion of $\{a \mid f(a) = g(a)\}$ satisfies the equalizer universal property.

Exercise 1.8. Construct two maps whose equalizer is the set of `CutRecord` values satisfying `FirstLiveOrdinal > SnapshotOrdinal`. You may change the codomain so that strict inequality becomes equality of encoded evidence.

Exercise 1.9. Explain why “every equalizer is monic” does not justify replacing equalizers by arbitrary injective functions in an unknown category.

Exercise 1.10. For the cospan $A \xrightarrow{f} C \xleftarrow{g} B$ in **Set**, prove that $\{(a, b) \mid f(a) = g(b)\}$ has the pullback universal property.

Exercise 1.11. Model an authorized subscription as a pullback. Define all four objects and both comparison maps; do not use the word “authorization” as an unexplained codomain.

Exercise 1.12. A snapshot has cut 20 and contains entities last changed at ordinals 4, 9, and 17. Explain why requiring `maxEntityOrdinal == snapshotOrdinal` is stronger than the usual soundness law. Give a pullback or subobject model for the weaker law.

Exercise 1.13. Design a counterexample execution in which entity application succeeds and projection-cursor advancement fails. Which local observations exist? Why is the intended global cone absent?

Exercise 1.14. Prove that an invariant $I : X \rightarrow Q$ is constant on the fibers of $r : X \rightarrow P$ if and only if there exists a unique function $\bar{I} : \text{im}(r) \rightarrow Q$ with $I = \bar{I} \circ r$ after restricting the codomain to the image.

Exercise 1.15. Give a `SessionStream` parameter set that is sufficient to decide an invariant but insufficient to reconstruct the whole global state.

Exercise 1.16. Write the evaluation and currying equations for a timeline projector whose explicit context is `(Event, SessionMetadata, TimelineView, SchemaVersion)`.

Exercise 1.17. Identify one hidden dependency that would invalidate your answer to Exercise 1.16. Repair the domain object.

Exercise 1.18. Show that a terminal object and pullbacks give binary products. Hint: form the pullback of the unique arrows $A \rightarrow 1 \leftarrow B$.

Exercise 1.19. Show that products and equalizers can construct the pullback of $A \xrightarrow{f} C \xleftarrow{g} B$ by equalizing $f \circ \pi_A$ and $g \circ \pi_B$ on $A \times B$.

Exercise 1.20 (chapter synthesis). Select one open `SessionStream` correctness obligation. Produce a one-page specification containing: the diagram, candidate cone, universal claim, counterexample schedule, and one implementation strategy that creates a stronger joint witness.

Chapter 2 - What Can Be Known from Each Context?

Chapter 1 asked for a single coherent witness. Real systems rarely expose that witness directly. They expose a family of partial views:

- the event store knows canonical events and event cursors;
- the timeline store knows materialized entities and snapshot cuts;
- a projector sees an event plus a pre-event view;
- a WebSocket connection sees a snapshot and selected live batches;
- an observer sees transport records, possibly with loss;
- an API caller sees only the supplied parameters and response.

The right object of study is therefore not one set of states. It is a **system of sets indexed by contexts**, together with lawful ways of forgetting information when we move to a poorer context.

That is the presheaf idea. To reach it carefully, we first need functors, natural transformations, and opposite categories.

2.1 From one category to another: functors

2.1.1 The need for structure-preserving interpretation

A timeline projection interprets canonical events as materialized state. A codec interprets typed values as bytes. A replay tool interprets event histories as final views. A trace analyzer interprets concrete execution records as abstract transitions.

Calling each of these “a function” says too little. The important question is whether interpretation respects identities and composition.

If history x is extended by history y , replay should satisfy a law of the form

$$R(xy) = R_y(R_x(S_0)),$$

not produce an unrelated state. If a codec path consists of encode followed by transport followed by decode, the interpretation of the composite path should be the composite of the interpretations.

2.1.2 Definition

Set example. A function between posets is structure-preserving when it is monotone. A group homomorphism is structure-preserving when it preserves identity and multiplication. A category packages many possible kinds of composition, and a functor preserves that categorical structure.

Definition 2.1 (Functor). A functor $F : \mathcal{C} \rightarrow \mathcal{D}$ assigns:

1. to each object A of $\mathcal{C}$, an object $F(A)$ of $\mathcal{D}$;
 2. to each arrow $f : A \rightarrow B$, an arrow $F(f) : F(A) \rightarrow F(B)$;
- such that

$$F(1_A) = 1_{F(A)}$$

and

$$F(g \circ f) = F(g) \circ F(f).$$

A functor therefore preserves domains, codomains, identities, and composites.

2.1.3 Example: replay as a functor

Define a category $\mathcal{H}_s$ for one session:

- objects are finite event prefixes $E_{\leq n}$;
- there is one arrow $E_{\leq m} \rightarrow E_{\leq n}$ when $m \leq n$;
- composition is prefix extension.

Define a category $\mathcal{T}_s$:

- objects are timeline states at cuts;
- arrows are state transitions induced by extending the history.

A replay interpretation $R : \mathcal{H}_s \rightarrow \mathcal{T}_s$ assigns a timeline state to each prefix and a transition to each prefix extension.

Functoriality says:

1. replaying an empty extension does nothing;
2. replaying from m to k through n equals replaying the composite extension from m to k .

If the projector reads an unrecorded clock, two replays of the same arrow can differ. Then the proposed R is not a well-defined functor on $\mathcal{H}_s$. The missing clock observation must be added to the source context, or removed from the projector.

2.1.4 Example: schema encoding

Let $\mathcal{V}$ have typed protobuf values as objects and explicit transformations between schema versions as arrows. Let $\mathcal{B}$ have byte representations and byte transformations as arrows. A serializer is functorial when it maps identity migrations to identity byte transformations and maps composed migrations to composed byte transformations.

This is stronger than “each individual value encodes.” It says whole transformation paths are coherent.

2.1.5 Example: forgetful functor

A **forgetful functor** discards structure while preserving the underlying arrows. For example, a functor may send a typed `TimelineEntity` to its raw payload bytes, forgetting semantic fields such as `Kind`, `Id`, and ordinal metadata.

Forgetting is often lawful and useful. It becomes dangerous when an engineer later assumes that the forgotten representation still determines the richer object. A functor need not be faithful to every distinction.

2.1.6 Full, faithful, and essentially surjective

These terms help describe how much an interpretation forgets.

Definition 2.2 (Faithful). A functor is faithful if distinct arrows in the source remain distinct after interpretation.

Definition 2.3 (Full). A functor is full if every arrow between interpreted objects in the target comes from an arrow in the source.

Definition 2.4 (Essentially surjective). A functor is essentially surjective if every target object is isomorphic to some object in its image.

SessionStream reading. A JSON encoding that maps two semantically distinct event variants to the same untagged object is not faithful. A browser representation may contain target-side mutations that have no corresponding canonical backend transformation, so the interpretation is not full. Neither failure is automatically bad, but both must be understood.

Student checkpoint. A functor does not merely map values. It maps a compositional world to another compositional world. Hidden dependencies usually mean the proposed source category is too small.

2.2 Natural transformations: coherent change across all contexts

2.2.1 The problem

Suppose there are two timeline projectors:

- F , the old projector;
- G , the new projector.

At each event prefix $E_{\leq n}$, we may have a migration

$$\eta_n : F(E_{\leq n}) \rightarrow G(E_{\leq n}).$$

A collection of per-prefix migrations is not enough. It should not matter whether we first extend the history and then migrate, or first migrate and then process the extension under the new projector.

2.2.2 Definition

Definition 2.5 (Natural transformation). Given functors $F, G : \mathcal{C} \rightarrow \mathcal{D}$, a natural transformation $\eta : F \Rightarrow G$ assigns to every object A an arrow

$$\eta_A : F(A) \rightarrow G(A)$$

such that for every arrow $f : A \rightarrow B$ in $\mathcal{C}$, the naturality square commutes:

$$G(f) \circ \eta_A = \eta_B \circ F(f).$$

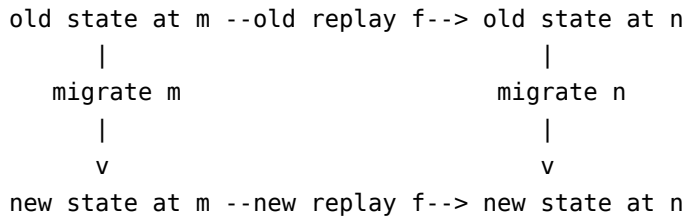
The arrows η_A are the **components** of η .

Definition 2.6 (Natural isomorphism). A natural transformation is a natural isomorphism when every component is an isomorphism.

SessionStream example. Suppose snapshots are represented at every cut both as sorted unique entity slices and as maps keyed by (Kind, Id) . If conversion at each cut is reversible and commutes with prefix restriction, the conversions form a natural isomorphism between the two snapshot presheaves. Reversible conversion at one cut alone is not enough.

2.2.3 SessionStream example: migration commutes with replay

Let $f : E_{\leq m} \rightarrow E_{\leq n}$ be a prefix extension. Naturality requires



This is a precise migration obligation. Testing only final states at one fixed prefix does not test naturality across all extensions.

2.2.4 SessionStream example: store implementations

Suppose F interprets abstract hydration operations using an in-memory store and G uses SQLite. A family of comparison maps between their states is useful only when it is natural with respect to Apply, Snapshot, and replay transitions.

An ad hoc comparison function that works after one scripted sequence may fail to commute with arbitrary operation sequences.

2.2.5 Counterexample: a context-sensitive migration

Suppose a migration assigns a missing timestamp using the current wall clock. Migrating a state at cut m today and later replaying to n can differ from replaying the old state to n and migrating tomorrow. The square fails because the components are not determined solely by the indexed object.

Again, the repair is not category-theory terminology. It is an engineering choice:

- record the timestamp as canonical data;
- supply a fixed migration clock as part of the context;
- or accept that the migration is not natural and version the resulting semantics explicitly.

2.2.6 Why naturality is a useful review question

For any system-wide change, ask:

1. What functors represent the old and new interpretations?
2. What is the component map at each context?
3. Which arrows in the source category must the migration commute with?
4. Can a test generate those arrows and compare both paths?

This question is stronger than “does the new implementation pass current tests?” and weaker than a full proof. It identifies the right family of commuting squares.

2.3 Opposite categories and why restriction reverses arrows

Presheaves are contravariant. The word sounds technical, but the behavior is familiar: more context can be restricted to less context.

2.3.1 Opposite category

Definition 2.7 (Opposite category). The opposite category $\mathcal{C}^{op}$ has the same objects as $\mathcal{C}$, but every arrow is reversed. If $f : A \rightarrow B$ in $\mathcal{C}$, then

$$f^{op} : B \rightarrow A$$

in $\mathcal{C}^{op}$. Composition is reversed accordingly.

2.3.2 Contravariant functor

Definition 2.8 (Contravariant functor). A contravariant functor from $\mathcal{C}$ to $\mathcal{D}$ is an ordinary functor

$$F : \mathcal{C}^{op} \rightarrow \mathcal{D}.$$

Equivalently, an arrow $f : A \rightarrow B$ in $\mathcal{C}$ induces an arrow

$$F(f) : F(B) \rightarrow F(A).$$

2.3.3 Why this matches information restriction

Let contexts be ordered by information inclusion:

$$V \subseteq U$$

means U contains at least as much observable information as V . There is an inclusion arrow

$$i : V \rightarrow U.$$

A state over the richer context U can be restricted to the poorer context V :

$$F(i) : F(U) \rightarrow F(V).$$

The direction reverses. The context arrow says “embed the smaller context into the larger one.” The data arrow says “forget from the larger observation down to the smaller one.”

2.3.4 SessionStream example: prefix truncation

If $m \leq n$, there is a context arrow

$$m \rightarrow n$$

meaning the prefix through m is included in the prefix through n . A contravariant trace assignment sends this to

$$\text{truncate}_{n,m} : F(n) \rightarrow F(m).$$

A full trace through n can be truncated to a trace through m .

The presheaf law will require

$$\text{truncate}_{n,n} = 1,$$

and for $k \leq m \leq n$,

$$\text{truncate}_{n,k} = \text{truncate}_{m,k} \circ \text{truncate}_{n,m}.$$

2.3.5 Counterexample: recomputation is not restriction

Suppose `restrict(n -> m)` replays the first m events using the **current** projector version rather than forgetting part of the already observed state. Then restricting directly from n

to k may differ from restricting n to m and then m to k . This is a new interpretation, not a restriction map for the original presheaf.

Restriction should preserve the meaning of the richer section while discarding coordinates. It should not silently rerun the world under new semantics.

2.4 Choosing the base category of contexts

A presheaf is only as good as its base category. The base category says what counts as a context and when one context refines another.

2.4.1 Context

Definition 2.9 (Context, in this book). A context is a declared boundary of available information or observation. A context arrow $V \rightarrow U$ means that V can be regarded as a subcontext, refinement, or probe of U .

This is a modeling definition, not a universal mathematical meaning of the word.

2.4.2 Prefix contexts

For one session, define $\mathcal{P}_s$ with objects natural-number cuts and arrows $m \rightarrow n$ when $m \leq n$.

This base captures time by prefix inclusion. It is simple and useful, but its nerve is essentially one-dimensional and contractible. Richer topological shape requires more than time alone.

2.4.3 Aspect contexts

Let aspects be

$$\{E, T, S, U, C, O\}$$

for event log, timeline state, snapshot, live UI, client state, and observer trace. A context may contain a subset of aspects. Inclusion of aspect sets gives context arrows.

A richer context such as $\{E, T, S\}$ restricts to $\{T, S\}$ by forgetting the raw event history.

2.4.4 Product contexts

Combine session, cut, and aspect:

$$U = (s, n, K).$$

A context arrow

$$(s, m, J) \rightarrow (s, n, K)$$

may exist when

$$m \leq n \quad \text{and} \quad J \subseteq K.$$

This produces a multidimensional base: temporal refinement and observational refinement can vary independently.

2.4.5 API-information contexts

For an endpoint, let a context be a subset of available parameters and trusted lookups. An arrow $P \rightarrow Q$ means $P \subseteq Q$. A section over Q is a locally lawful assignment to those coordinates. Restriction forgets parameters.

This base is useful for parameter sufficiency, privacy, and least-authority analysis.

2.4.6 Trace-region contexts

For concurrent executions, a context can be a downward-closed region of a happens-before partial order. Restriction removes events outside the region. This base is more faithful than a single total ordinal when operations can be concurrent.

2.4.7 Choosing arrows carefully

An arrow in the base is a promise that restriction is meaningful. Do not add an arrow merely because one struct can be converted to another.

For every proposed context arrow $V \rightarrow U$, ask:

- Is V genuinely less informative than U ?
- Can every lawful observation over U be restricted to one over V ?
- Is the restriction deterministic?
- Do identity and composition laws hold?
- Does the restriction preserve the semantics being studied?

2.5 Presheaves: sets of sections indexed by context

2.5.1 Motivation

We now have:

- a category of contexts;
- for each context, a set of locally possible observations;
- for each refinement, a restriction operation.

This is exactly a presheaf.

2.5.2 Definition

Definition 2.10 (Presheaf). A presheaf of sets on a category $\mathcal{C}$ is a functor

$$F : \mathcal{C}^{op} \rightarrow \mathbf{Set}.$$

Thus:

1. each context U receives a set $F(U)$;
2. each arrow $f : V \rightarrow U$ receives a restriction function

$$F(f) : F(U) \rightarrow F(V);$$

3. identity and composition are preserved.

Definition 2.11 (Section). An element $s \in F(U)$ is a section of F over U .

Definition 2.12 (Restriction). For $f : V \rightarrow U$, the section $F(f)(s)$ is the restriction of s along f , often written $s|_V$ when the arrow is understood.

Definition 2.13 (Global section). If the base has a maximal or whole-system context X , an element of $F(X)$ is a global section. More generally, let 1 be the **terminal presheaf**, which assigns a one-element set to every context and the only possible restriction map to every arrow. A global section is a natural transformation $1 \Rightarrow F$.

The second definition means a compatible choice of one section at every context. It matters when there is no single largest context, but the first definition is sufficient for many engineering models.

2.5.3 Presheaf laws in software form

For every context U :

`Restrict(U, U, s) == s`

For $W \rightarrow V \rightarrow U$:

`Restrict(U, W, s) == Restrict(V, W, Restrict(U, V, s))`

These are property-test candidates. A restriction API without these laws is not a presheaf restriction.

2.5.4 Example: raw event traces

Let $F(n)$ be all canonical event histories through cut n that satisfy schema and ordinal laws. Restriction $F(n) \rightarrow F(m)$ truncates after ordinal m .

This is a presheaf on prefix contexts.

A global section through final cut N is one lawful complete event trace. The presheaf itself includes all possible traces at all cuts.

2.5.5 Example: timeline observations

Let $T(n, K)$ be the set of all locally lawful timeline observations at cut n exposing fields in aspect set K . Restrictions can:

- forget later history;

- forget payload fields;
- forget cursor evidence;
- forget tombstone history;
- forget observer metadata.

A restriction is lawful only when the poorer observation can be obtained without inventing new data or changing semantics.

2.5.6 Example: invariant presheaf

Let $\text{Law}(U)$ be the set of predicates decidable from context U . Restriction is subtle: a predicate on a rich context does not always descend to a poorer one.

A safer presheaf assigns to U the predicates **already known to be stable under forgetting beyond** U . Then restriction maps a predicate to its weaker-context form when such descent is part of the data.

This example warns that “all predicates” is not automatically a presheaf under naive forgetting.

2.5.7 Example: completion presheaf

Let $\text{Comp}(U)$ be the set of global executions consistent with a local observation over U . If $V \rightarrow U$ forgets information, then every completion of the richer observation is also a completion of its restriction. But forgetting a local observation usually *enlarges* its set of completions, so the most obvious map goes in the opposite direction from an ordinary presheaf restriction.

This is an important modeling boundary: not every context-indexed construction is contravariant in the same way. Before calling something a presheaf, write the induced map for one context arrow and check its direction.

2.5.8 Example: schema knowledge

Let $\text{Schema}(U)$ contain the event and entity schemas visible to context U . Restriction forgets descriptors not visible in the poorer context. A browser with only a subset of descriptors may be unable to decode an Any value even though the server can.

A global schema section would assign compatible descriptors across all participating runtimes.

2.6 Natural transformations between presheaves

Presheaves form a category. Its objects are presheaves; its arrows are natural transformations.

2.6.1 Definition in restriction language

Let $F, G : \mathcal{C}^{op} \rightarrow \mathbf{Set}$. A natural transformation $\eta : F \Rightarrow G$ assigns a function

$$\eta_U : F(U) \rightarrow G(U)$$

for every context U , such that for every $V \rightarrow U$,

$$\eta_V(s|_V) = \eta_U(s)|_V.$$

Translate first, then restrict; or restrict, then translate. The result must be the same.

2.6.2 SessionStream example: trace summarization

Let $F(U)$ be detailed observer traces and $G(U)$ be summary metrics. A summary transformation is natural if summarizing a trace and then restricting the summary equals restricting the trace and summarizing the smaller region.

A metric such as “total events since process start” may not be natural under arbitrary trace-region restriction because the restricted context lacks the same origin. A metric such as “count of records in this region” is natural.

2.6.3 SessionStream example: redaction

Let $F(U)$ contain full payload observations and $G(U)$ contain redacted observations. Redaction is natural when redacting before or after forgetting fields gives the same result.

Context-dependent redaction policies can still be modeled, but the components and base arrows must include the policy context. Otherwise naturality may fail silently.

2.7 Fibers, missing coordinates, and parameter sufficiency

Presheaf language organizes contexts. Fibers answer the original API question more directly.

2.7.1 Fiber

Definition 2.14 (Fiber of a function). For $r : X \rightarrow P$ and $p \in P$, the fiber over p is

$$r^{-1}(p) = \{x \in X \mid r(x) = p\}.$$

Think of r as restriction from full semantic state to exposed parameters.

2.7.2 Three meanings of “enough”

Parameters p can be enough in at least three different senses.

1. **Existence sufficiency:** the fiber is nonempty.
2. **Reconstruction sufficiency:** the fiber has exactly one element.
3. **Invariant sufficiency:** the target invariant has one value across the fiber.

These should not be conflated.

2.7.3 SessionStream example: event identity

Suppose an event is identified only by $(\text{SessionId}, \text{Ordinal})$. A bus redelivery that receives a fresh ordinal lies in a different fiber of that parameter map even if it is the same logical event.

The problem is usually not a cohomological obstruction. The base coordinates omit a stable logical `EventId` or producer message identity.

Add the coordinate, then restate the restriction map.

2.7.4 SessionStream example: deterministic projection

Suppose declared projector input is

$$P = (\text{event}, \text{session}, \text{timelineView}).$$

If output also depends on wall-clock time, then many full execution states restrict to the same P but produce different outputs. The output is not constant on fibers, so the declared parameters are insufficient.

Repairs:

- add the clock observation to P ;
- convert the observation into a canonical event;
- remove the dependency;
- or weaken the promised invariant.

2.7.5 Missing coordinate versus failed gluing

Use this diagnostic order:

1. Are the contexts and coordinates complete enough to state the invariant?
2. Are restriction maps well-defined and functorial?
3. Are the chosen local sections pairwise compatible?
4. Does compatible local data glue?
5. Only then ask whether a higher obstruction remains.

Many software bugs stop at step 1 or 2.

2.8 Representables and the Yoneda point of view

This section is optional on a first reading, but it gives a rigorous form to the idea “know an object by how every context can probe it.”

2.8.1 Representable presheaf

For objects U, A of $\mathcal{C}$, the collection of arrows $U \rightarrow A$ is called the **hom-set** and is written $\mathcal{C}(U, A)$. For a fixed object A , define

$$yA(U) = \mathcal{C}(U, A).$$

A section of yA over U is an arrow $U \rightarrow A$: a way for context U to probe or instantiate A . Restriction along $V \rightarrow U$ is precomposition.

Definition 2.15 (Representable presheaf). A presheaf is representable if it is naturally isomorphic to yA for some A .

2.8.2 Yoneda lemma, practical statement

Theorem 2.16 (Yoneda lemma, informal working form). Natural transformations from yA to a presheaf F correspond exactly to sections of F over A :

$$\text{Nat}(yA, F) \cong F(A).$$

Here $\text{Nat}(yA, F)$ denotes the set of natural transformations from yA to F .

An element is completely characterized by how it acts under every probe.

2.8.3 SessionStream reading

A Snapshot object can be studied by all allowed requests, codecs, projections, and observer probes into it. If two snapshot implementations respond identically to every probe in the chosen category, Yoneda says they are indistinguishable at that categorical level.

The phrase “chosen category” remains crucial. If timing, allocation, or stable iteration order matters, those probes must be included.

2.8.4 Engineering use

Yoneda suggests interface-oriented test design:

- define the probes that constitute observable behavior;
- compare implementations through all generated probes;
- avoid relying on hidden representation equality.

This is not a free theorem about arbitrary production code. It is a disciplined way to specify the observation category.

2.9 The presheaf category as a software universe

For a fixed small base category $\mathcal{C}$, presheaves and natural transformations form a category

$$\mathbf{Set}^{\mathcal{C}^{op}}.$$

Chapter 3 will define a **topos** formally as a category with finite limits, exponentials, and a subobject classifier. The presheaf category is the principal context-dependent example. It has rich structure:

- limits and colimits are computed context by context;
- exponentials exist;
- there is a subobject classifier;
- therefore it is a topos.

This will matter in Chapter 3. For now, the key practical point is that a whole family of context-dependent values can be treated as one mathematical object, and transformations between such families can be checked for naturality.

2.9.1 Pointwise limits

Given presheaves F and G , their product is

$$(F \times G)(U) = F(U) \times G(U)$$

with restriction performed componentwise.

Equalizers are also computed at each context. This lets us build presheaves of locally lawful combined observations from simpler presheaves.

2.9.2 SessionStream application

Let

- $E(U)$ be event evidence visible at U ;
- $T(U)$ be timeline evidence visible at U ;
- $K(U)$ be cursor evidence visible at U .

A presheaf of combined observations can be built by products. A presheaf of observations satisfying cursor equations can be built by equalizers. Chapter 3 will then ask whether those local lawful observations glue across covers.

2.10 Chapter laboratory: define a SessionStream presheaf

Build one explicit presheaf. Recommended choices:

- snapshot observations by cut and aspect;
- transport traces by time region;
- schema availability by runtime;
- API parameter assignments by parameter subset;
- projection evidence by session and ordinal.

Your specification must include:

1. the base category's objects;
2. its arrows;
3. the set $F(U)$ for every kind of object;
4. the restriction function for every kind of arrow;
5. an identity-law argument;
6. a composition-law argument;

7. two concrete sections;
8. one invalid would-be restriction;
9. one natural transformation out of your presheaf;
10. one fiber-based sufficiency question.

Then implement property tests for identity and composition.

2.11 Exercises

Exercise 2.1. Define the identity functor on the prefix category and verify both functor laws.

Exercise 2.2. Define a replay functor for a deterministic counter projection. Its events are `Increment(k)` and its state is an integer. Specify object and arrow mappings.

Exercise 2.3. Add a hidden random choice to Exercise 2.2. Explain exactly why the mapping is no longer a well-defined functor on the original source category.

Exercise 2.4. Give one faithful but not full software interpretation and one full but not faithful set-theoretic functor.

Exercise 2.5. Write the naturality square for a migration between two versions of `ChatMessageEntity` over prefix extensions.

Exercise 2.6. Construct a migration component that works at every individual prefix but fails naturality. Give explicit values on a two-step history.

Exercise 2.7. Prove that the composite of two natural transformations is natural.

Exercise 2.8. Construct the opposite of the category $0 \rightarrow 1 \rightarrow 2$. List every arrow and composite.

Exercise 2.9. Explain why a presheaf on prefix contexts sends the extension arrow $m \rightarrow n$ to a truncation map $F(n) \rightarrow F(m)$.

Exercise 2.10. Write a Go-like interface for a finite presheaf. Include enough information to validate composable restrictions.

Exercise 2.11. Design a mutation that violates the presheaf composition law but still passes identity tests.

Exercise 2.12. Define a presheaf of event names visible to each runtime: Go server, Goja host, WebSocket browser. What are the context arrows? What does restriction do?

Exercise 2.13. Is “set of possible global completions” naturally covariant or contravariant under forgetting context? Work through a two-parameter example and justify your answer.

Exercise 2.14. Let $r : X \rightarrow P$ forget a schema version. Give a fiber containing two full states that produce different projection results. Which sufficiency notion fails?

Exercise 2.15. Prove that an invariant descends to exposed parameters exactly when it is constant on fibers, using the image of r as codomain.

Exercise 2.16. Find a `SessionStream` law for which parameters are sufficient to decide the law but not sufficient to reconstruct the event history.

Exercise 2.17. Define the representable presheaf yA on a three-object poset. Compute its values and restriction maps explicitly.

Exercise 2.18. State the Yoneda correspondence for one section of your presheaf from the chapter laboratory. Describe the associated natural transformation.

Exercise 2.19. Show pointwise that the product of two presheaves satisfies the presheaf identity and composition laws.

Exercise 2.20 (chapter synthesis). Produce a context diagram for `SessionStream` with at least two independent dimensions, define a presheaf on it, and classify one real bug as: missing coordinate, non-functorial restriction, incompatible sections, or unresolved gluing problem. Defend the classification.

Chapter 3 - When Do Compatible Local Views Determine a Whole?

A presheaf tells us what can be observed in each context and how richer observations restrict to poorer ones. It does **not** yet say that local observations can be assembled into a global one.

That extra local-to-global principle is the sheaf condition.

The central example is SessionStream hydration. A reconnecting client receives:

- a snapshot representing a past event prefix;
- buffered and future UI events representing a later suffix.

The two pieces overlap at a cut. If they agree there, can they be assembled into one client execution? Is the assembly unique? What system assumptions make the answer yes?

Goldblatt motivates sheaves in a similar concrete-to-abstract order. He begins with local sections over open sets, asks when compatible sections can be pasted, isolates the compatibility-and-unique-pasting condition, and then generalizes open covers to categorical sites. We will follow that path, using execution contexts in place of geographical regions.

3.1 Local pieces are not automatically a whole

Consider three observations:

A: event store contains events through ordinal 50

B: timeline projection cursor is 50

C: snapshot contains entities through ordinal 49

Observations A and B agree on the number 50. Observation C may be valid if event 50 has no timeline effect, or invalid if it should have one. Pairwise comparison of raw ordinals does not settle the question. We need:

1. a declared notion of **coverage** - which local contexts collectively count as observing the whole;
2. a declared **overlap** for each pair of contexts;
3. restriction maps to those overlaps;
4. a definition of compatibility;
5. a theorem or protocol stating when compatible data has a unique global assembly.

The sheaf condition is therefore not “all local data eventually becomes consistent.” It is a precise statement relative to a chosen base category, a chosen coverage policy, and a chosen presheaf.

3.2 Covers: what counts as enough local observation?

3.2.1 Open-cover motivation

For an ordinary topological space, an open cover of a region U is a family of open sets $\{U_i\}$ whose union is U .

A function on U can be described by functions on each U_i if the local functions agree on every overlap $U_i \cap U_j$.

Software has no automatic union operation on contexts. We must declare which families count as covers.

3.2.2 Cover

Definition 3.1 (Cover, working form). A cover of a context U is a declared family of context arrows

$$\{u_i : U_i \rightarrow U\}_{i \in I}$$

that is considered jointly sufficient to observe or reconstruct the aspect of U being studied.

Coverage is part of the model. It is not inferred merely from names such as “service,” “endpoint,” or “database.”

3.2.3 Temporal cover

Let $U_{\leq m}$ be a client execution through ordinal m . Choose a cut $n < m$ and define:

- P_n : the past context through n ;
- $Q_{n,m}$: a suffix context containing a boundary state at n and outputs for $(n, m]$;
- B_n : the boundary context at cut n .

Then P_n and $Q_{n,m}$ cover $U_{\leq m}$ when the model declares that a sound past state plus a complete ordered continuation determines the execution through m .

Notice that a raw list of UI events after n may not be enough. The suffix context needs whatever boundary data the reducer requires. Coverage forces us to state that dependency.

3.2.4 Aspect cover

Let a whole diagnostic context contain event evidence, timeline evidence, cursor evidence, and transport evidence. A proposed cover might be:

$$U_E = \{event, cursor\},$$

$$U_T = \{timeline, cursor\},$$

$$U_W = \{snapshot, live, cursor\}.$$

Their overlaps include cursor claims. This family covers the whole only if those aspects collectively determine the invariant being checked. It might cover ordering correctness while failing to cover authorization correctness.

3.2.5 API parameter cover

Suppose an operation's semantic context is

$$U = \{order, priceVersion, quantity, currency, account\}.$$

Two endpoint contexts might be

$$U_1 = \{order, priceVersion, quantity\},$$

$$U_2 = \{order, currency, account\}.$$

They cover U only if their union contains every coordinate and their overlap on order supplies enough identity to join the values. If `priceVersion` is implicitly account-dependent, the cover may be false despite field coverage.

3.3 Pretopologies and sites: lawful coverage policies

A one-off cover is useful. A theory of local reasoning needs coverage to behave predictably under identity, refinement, and composition.

3.3.1 Pulling back a cover

Suppose $\{U_i \rightarrow U\}$ covers U , and $V \rightarrow U$ is a refined context. We expect the portions of U_i visible inside V to cover V .

Categorically, those portions are pullbacks:

$$\begin{array}{ccc} V \times_U U_i & \dashrightarrow & U_i \\ | & & | \\ v & & v \\ V & \dashrightarrow & U \end{array}$$

This is why Chapter 1 placed so much emphasis on pullbacks. Local coverage should survive change of context.

3.3.2 Definition of pretopology

Definition 3.2 (Pretopology). A pretopology on a category $\mathcal{C}$ assigns to each object U a collection of covering families $\{U_i \rightarrow U\}$ such that:

1. **identity coverage:** $\{1_U : U \rightarrow U\}$ covers U ;
2. **composition of covers:** if $\{U_i \rightarrow U\}$ covers U and each U_i is covered by $\{V_{ij} \rightarrow U_i\}$, then the composites $\{V_{ij} \rightarrow U\}$ cover U ;
3. **pullback stability:** pulling a cover back along any arrow $V \rightarrow U$ gives a cover of V .

Definition 3.3 (Site). A category equipped with a declared local-coverage structure is called a site. In this book the structure will usually be a pretopology. Section 3.9 gives the equivalent sieve-based notion called a Grothendieck topology.

The site says what “local” means.

3.3.3 Why each axiom matters in software

Identity coverage. A complete observation should count as a trivial local description of itself.

Composition. If a snapshot is covered by per-entity views, and each entity view is covered by schema and payload views, then the combined smaller views should still cover the snapshot.

Pullback stability. If a diagnostic policy covers all sessions, restricting to one tenant or one time window should still give a valid local coverage policy. This is a least-authority sanity condition: localizing the question should not destroy the meaning of coverage.

3.3.4 A non-example

Suppose a team declares that “the production database and the metrics dashboard cover system state.” Pull the claim back to a single request. The dashboard may contain only aggregate counts and no request-level evidence, so the pulled-back family does not cover the request context. The original coverage claim is too coarse to form the intended site.

3.3.5 Grothendieck topology and sieve preview

A pretopology describes covers by families. A **sieve** packages a family together with every further refinement of its members. A Grothendieck topology declares certain sieves covering. We will define both notions fully in Section 3.9 because sieves also become truth values when the presheaf category is viewed as a topos.

For most engineering models, an explicit finite pretopology is the easiest place to start.

3.4 Matching families and the sheaf condition

3.4.1 Compatibility on overlaps

Let F be a presheaf on a site and let $\{u_i : U_i \rightarrow U\}$ cover U . Choose local sections

$$s_i \in F(U_i).$$

For each pair, form the pullback overlap

$$U_{ij} = U_i \times_U U_j.$$

The two local sections restrict to $F(U_{ij})$.

Definition 3.4 (Matching family). The family (s_i) is matching, or compatible, if for every i, j the two restrictions agree on U_{ij} .

Symbolically,

$$s_i|_{U_{ij}} = s_j|_{U_{ij}}.$$

Pairwise compatibility is not vague agreement. It is equality after applying specified restriction maps to a specified overlap.

3.4.2 Amalgamation

Definition 3.5 (Amalgamation). An amalgamation of a matching family (s_i) is a section

$$s \in F(U)$$

whose restriction to each U_i is s_i .

The section s is the proposed whole assembled from the local pieces.

3.4.3 Sheaf

Definition 3.6 (Sheaf). A presheaf F on a site is a sheaf if every matching family on every cover has exactly one amalgamation.

The condition has two logically distinct parts:

1. **existence:** compatible local sections glue to at least one global section;
2. **uniqueness:** they glue to at most one global section.

3.4.4 Separated presheaf

Definition 3.7 (Separated presheaf). A presheaf is separated if matching local sections have at most one amalgamation.

A separated presheaf may fail existence but not uniqueness.

Software reading. If local observations uniquely determine a global result whenever a result exists, but some compatible families have no realizable global execution, the model is separated but not a sheaf.

3.4.5 Ordinary function example

Let $F(U)$ be the set of functions from a region U to a fixed set X , with ordinary restriction. Compatible local functions paste pointwise to a unique global function. Therefore F is a sheaf on the usual open-cover site.

This is the model to hold in mind: agreement on overlap makes the pointwise definition independent of which local region is chosen.

3.4.6 A failing presheaf: bounded summaries

Let $F(U)$ be summaries of data in U containing only (count, maximum). Two local summaries can agree on overlap summaries but still fail to determine a unique global summary if duplicates across regions are not tracked. The presheaf may lose too much information for unique gluing.

The failure is not in the cover. It is in the chosen sections and restrictions.

3.4.7 A failing presheaf: arbitrary local IDs

Suppose each local context assigns fresh IDs to the same semantic event. Local sections may agree on all payload fields yet yield multiple global identity assignments. Uniqueness fails. Canonical identity or explicit transition maps are needed.

3.5 The equalizer form of the sheaf condition

For a finite cover, the sheaf condition is itself a limit statement.

3.5.1 Two-piece cover

Let U be covered by U_1, U_2 . There are two restriction maps

$$F(U_1) \times F(U_2) \rightrightarrows F(U_1 \times_U U_2).$$

The first restricts the U_1 section to the overlap; the second restricts the U_2 section.

Compatible pairs form the equalizer of those maps. The sheaf condition says the restriction map

$$F(U) \rightarrow \text{Eq}(F(U_1) \times F(U_2) \rightrightarrows F(U_1 \times_U U_2))$$

is a bijection.

This is the bridge from Goldblatt's limits to sheaves:

A global section is exactly a compatible tuple of local sections.

3.5.2 General cover

For a family $\{U_i \rightarrow U\}$, write

$$C^0 = \prod_i F(U_i)$$

and

$$C^1 = \prod_{i,j} F(U_i \times_U U_j).$$

There are two maps $C^0 \rightrightarrows C^1$ taking each family to its two overlap restrictions. The matching families form their equalizer. A sheaf identifies $F(U)$ with that equalizer.

The notation C^0, C^1 anticipates cochains, but no subtraction or cohomology is being used yet.

3.5.3 Engineering test

For a finite model, implement:

```
GlobalSections(U)
  -> LocalSections(U_i)
  -> OverlapRestrictions(U_i x_U U_j)
```

Then test:

- every global section yields a matching family;
- every generated matching family has a reconstruction;
- reconstruction restricts back to the original family;
- two reconstructions of the same family are equal.

This is an executable finite sheaf test.

3.6 Hydration as a gluing theorem

We now formulate the running protocol carefully enough to apply the sheaf definition.

3.6.1 The semantic reducer

Let C_n be client state through ordinal n . Let a deterministic client reducer be

$$r : C \times UEvent \rightarrow C.$$

For an ordered word w of UI events, write

$$\text{fold}(c, w)$$

for repeated reduction.

A snapshot at cut n carries a state S_n intended to equal C_n for the durable aspects represented by hydration.

3.6.2 The contexts

Define:

- P_n : past through cut n ;
- $Q_{n,m}$: boundary state at n plus an ordered UI suffix through m ;
- B_n : boundary facts at cut n ;
- U_m : complete client reconstruction through m .

The cover is

$$P_n \rightarrow U_m, \quad Q_{n,m} \rightarrow U_m.$$

Their overlap is B_n .

3.6.3 Sections

A section over P_n includes:

- session identity;
- snapshot ordinal n ;
- snapshot entities or reconstructed client state;
- schema version and reducer version needed to interpret it.

A section over $Q_{n,m}$ includes:

- the same session identity;
- declared starting cut n ;
- ordered UI batches with ordinals greater than n ;
- enough boundary state to apply the reducer.

A section over B_n contains the common boundary information: session, cut, schema/reducer identity, and perhaps a digest of the state at n .

3.6.4 Matching condition

The snapshot section and suffix section match when their restrictions to B_n agree. At minimum:

$$\text{session}(S_n) = \text{session}(L),$$

$$\text{cut}(S_n) = \text{start}(L) = n,$$

and their schema/reducer interpretations agree.

If the suffix contains a state digest at the boundary, that digest must equal the snapshot digest.

3.6.5 Amalgamation

The proposed amalgamation is

$$C_m = \text{fold}(S_n, L_{(n,m]}).$$

Existence requires:

- the snapshot is sound at n ;
- every required post-registration event is either represented in the snapshot or present in the suffix;
- events at or before n are not reapplied;
- suffix events are ordered;
- the reducer is defined for all payloads and schema versions;
- overflow or decode failures are explicit rather than silently omitted.

Uniqueness requires deterministic reduction and stable interpretation.

3.6.6 Proposition: snapshot-plus-suffix gluing

Proposition 3.8. Assume:

1. $S_n = C_n$;
2. the suffix contains exactly the client-visible outputs attributable to events $n + 1$ through m , in ordinal order;
3. the reducer is deterministic;
4. event interpretation and schemas are fixed;
5. duplicate outputs at ordinals at or before n are removed.

Then S_n and $L_{(n,m]}$ have a unique amalgamation C_m given by folding the suffix over the snapshot.

Proof. Existence follows by applying the deterministic reducer successively to the ordered suffix. The result extends the past section because it begins at S_n . It extends the suffix section by construction. For uniqueness, any amalgamation must begin at the same boundary state and apply the same ordered transitions under the same deterministic reducer. Induction on suffix length forces equality at every ordinal. $\square$

3.6.7 Mapping to the WebSocket implementation

The SessionStream WebSocket adapter registers a subscription in a hydrating state before loading the snapshot. Concurrent UI batches are buffered. After the snapshot is sent, buffered batches at or before the snapshot ordinal are filtered, later batches are sorted and sent, late buffered batches are flushed while the subscription lock prevents overtaking, and only then is the subscription marked live.

Each step exists to protect a premise of Proposition 3.8.

- hydrating registration protects completeness after the registration point;
- filtering protects overlap compatibility;
- sorting protects ordered reduction;
- locked late flush protects uniqueness of the observed sequence;
- overflow produces an explicit failure instead of a false global section.

3.6.8 Buffer overflow is an honest no-amalgamation result

When the hydration buffer exceeds its bound, the system cannot guarantee that the observed snapshot and retained suffix cover the whole execution. Closing the connection and forcing rehydration is mathematically honest: the current matching family is incomplete, so the protocol refuses to fabricate an amalgamation.

3.7 Distinguishing local-to-global failures

A disciplined diagnosis names which part of the sheaf story failed.

3.7.1 Bad cover

The selected local contexts never contained enough information. Example: snapshot plus UI events cannot reconstruct a timeline-only field that is neither in the snapshot nor derivable from UI events.

Repair: change the coverage policy or the promised whole.

3.7.2 Ill-defined restriction

A supposedly poorer observation is recomputed with new semantics instead of being restricted. Identity or composition laws fail.

Repair: define genuine forgetting maps or enlarge the context with version coordinates.

3.7.3 Non-matching family

Local sections disagree on overlap. Example: snapshot declares schema version 2 while suffix payloads are decoded under version 3.

Repair: reject, migrate, or negotiate until the boundary agrees.

3.7.4 No amalgamation

The local pieces agree on recorded overlaps but no global execution realizes them. This can happen when pairwise overlaps omit a higher-order transactional constraint.

Repair: add the missing joint context, strengthen the cover, or represent the transaction state explicitly.

3.7.5 Multiple amalgamations

The local pieces underdetermine identity or ordering, so several global executions fit.

Repair: add stable IDs, ordering evidence, or stronger overlap data. This is a failure of separatedness.

3.7.6 Hidden coordinate

The context omitted a relevant input such as clock, schema version, or producer identity.

Repair: enlarge the base before discussing sheaf failure.

3.8 Sheafification: repairing a presheaf universally

3.8.1 Motivation

A presheaf may have useful local data but fail gluing. Mathematics provides a canonical construction that converts it into a sheaf while preserving its local information as universally as possible.

3.8.2 Definition

Definition 3.9 (Sheafification, universal form). A sheafification of a presheaf F is a sheaf aF together with a natural map

$$\eta : F \rightarrow aF$$

such that every natural map from F to any sheaf G factors uniquely through η :

$$F \xrightarrow{\eta} aF \rightarrow G.$$

Sheafification is not arbitrary cleanup. It is universal among maps from F into sheaves.

3.8.3 Software analogies

The following repairs can resemble parts of sheafification, but none is automatically the mathematical sheafification of a production system.

Canonicalization. Identify local representations that agree after sufficient refinement.

Completion. Add global objects represented by compatible local families.

Boundary enrichment. Add missing overlap data such as schema versions, state digests, or stable IDs.

Joint witnessing. Add a transaction or atomic protocol state that realizes previously separate observations together.

Coverage correction. Stop declaring a family a cover when it is insufficient.

3.8.4 Separated reflection before full sheafification

A common two-step repair is:

1. enforce uniqueness by identifying locally indistinguishable sections;
2. add amalgamations for compatible families.

In software terms: canonicalize identity first, then add missing global assemblies. Retry identity is a good example: without stable event identity, even locally identical deliveries may produce multiple global histories.

3.9 Toposes: a universe for context-dependent sets

Sheaves are not merely isolated objects. Sheaves on a suitable site form a category with much of the structural richness of **Set**.

3.9.1 Elementary topos

Definition 3.10 (Elementary topos). An elementary topos is a category with finite limits, exponentials, and a subobject classifier.

Some presentations include finite colimits, which follow from the other conditions in an elementary topos. The concise definition is enough here.

Examples.

- **Set** is a topos.
- For a small category $\mathcal{C}$, the presheaf category $\mathbf{Set}^{\mathcal{C}^{op}}$ is a topos.
- Sheaves of sets on a site form a **Grothendieck topos**, meaning a category equivalent to the category of sheaves of sets on some site.

3.9.2 Why software engineers should care

A topos provides, internally:

- finite joint witnesses;
- function objects;
- predicates as subobjects;
- truth values appropriate to context;
- logical operations and quantifiers;
- a setting where local data is treated as ordinary data of that universe.

The claim is not that every software architecture should be implemented “inside a topos.” The value is conceptual: once data is context-indexed, predicates and functions should be context-indexed too. The phrase **internal logic** means the logical language interpreted using the objects, arrows, subobjects, and truth values available inside that category, rather than by stepping outside to ordinary sets for every assertion.

3.9.3 Subobject

Definition 3.11 (Subobject). A subobject of A is an equivalence class of monic arrows $m : S \hookrightarrow A$, where two monics represent the same subobject when their domains are isomorphic over A .

In **Set**, subobjects correspond to subsets.

SessionStream example. Sound snapshots form a subobject of all field-typed snapshots. Authorized subscriptions form a subobject of all syntactically valid subscription requests.

3.9.4 Subobject classifier

In **Set**, a subset $S \subseteq A$ has a characteristic function

$$\chi_S : A \rightarrow \{0, 1\}$$

that returns true exactly on S .

Definition 3.12 (Subobject classifier). A subobject classifier in $\mathcal{E}$ is an object Ω with an arrow

$$\text{true} : 1 \rightarrow \Omega$$

such that every monic $m : S \hookrightarrow A$ is a pullback of true along a unique characteristic arrow

$$\chi_m : A \rightarrow \Omega.$$

This makes predicates into arrows.

3.9.5 Truth values are contextual

In a presheaf topos, $\Omega(U)$ is not generally just $\{\text{false}, \text{true}\}$. Its elements are sieves on U .

Definition 3.13 (Sieve). A sieve S on U is a collection of arrows with codomain U that is closed under precomposition: if $f : V \rightarrow U$ is in S and $g : W \rightarrow V$ is any arrow, then $f \circ g : W \rightarrow U$ is in S .

A sieve records all refinements under which a proposition is verified.

The **maximal sieve** on U contains every arrow with codomain U . Given a sieve S on U and an arrow $f : V \rightarrow U$, its pullback sieve f^*S contains those arrows $g : W \rightarrow V$ for which $f \circ g$ lies in S .

Definition 3.14 (Grothendieck topology). A Grothendieck topology J assigns to each context U a collection $J(U)$ of covering sieves satisfying:

1. the maximal sieve on U covers U ;
2. if S covers U , then f^*S covers V for every $f : V \rightarrow U$;
3. if S covers U and a sieve R has f^*R covering the domain of every f in S , then R covers U .

These are the sieve versions of identity, pullback stability, and composition of covers. The open-cover pretopology and the temporal covers used for hydration generate Grothendieck topologies by closing their covering families under further refinement.

SessionStream example. At a whole diagnostic context U , the proposition “the snapshot is sound” may be verified only by refinements that include both the cut and entity ordinals. The truth value is the sieve of all probes carrying enough evidence and satisfying the law.

3.9.6 Covering sieve

A sieve is covering when it belongs to $J(U)$. Intuitively, it contains enough local probes to count as evidence over U . A pretopology generates such covering sieves by taking every arrow that factors through some member of a declared covering family.

This connects coverage and truth: a proposition holds locally over a cover when the refinements that verify it form a covering family or covering sieve.

3.10 Local truth and Kripke-Joyal reasoning

3.10.1 Intuitionistic logic and forcing notation

Definition (Intuitionistic logic, working description). Intuitionistic logic does not assume that every proposition is either proved or refuted. A proof of existence must supply a witness, and a proof of disjunction must justify an alternative, possibly only after passing to a local cover. Topos and sheaf semantics naturally support this evidence-sensitive logic.

Write

$$U \Vdash \varphi$$

for “the proposition φ holds at context U .”

This is called a forcing or Kripke-Joyal judgment. Its meaning depends on the topos and interpretation.

3.10.2 Persistence under refinement

If $U \Vdash \varphi$ and $V \rightarrow U$ is a refinement, then typically

$$V \Vdash \varphi|_V.$$

Truth persists when more local detail is examined, provided the proposition is restricted appropriately.

3.10.3 Conjunction

$$U \Vdash \varphi \wedge \psi$$

when both $U \Vdash \varphi$ and $U \Vdash \psi$.

Example. At a hydration context, both snapshot soundness and suffix ordering hold.

3.10.4 Disjunction

In sheaf semantics, $U \Vdash \varphi \vee \psi$ can mean there is a cover $\{U_i \rightarrow U\}$ such that on each U_i , one of the alternatives holds. The same alternative need not hold uniformly everywhere.

Software example. Every partition of a batch may be processed either from cache or by recomputation, but no global single branch describes the whole batch.

3.10.5 Existential quantification

$U \Vdash \exists x \varphi(x)$ can mean that there is a cover $\{U_i \rightarrow U\}$ and local witnesses x_i on each U_i satisfying φ .

A global witness need not exist even when witnesses exist locally.

SessionStream example. Each transport shard may have a local event identifier that explains a record, while no globally stable event ID exists across shards. Local existence is weaker than global identity.

3.10.6 Implication and universal quantification

Implication is tested under all refinements:

$$U \Vdash \varphi \Rightarrow \psi$$

when every refinement $V \rightarrow U$ that forces φ also forces ψ .

Universal quantification similarly requires the property for every refinement and every local element in that refinement.

3.10.7 Why excluded middle can fail

At a partial context, neither φ nor $\neg\varphi$ may be justified. For example, a timeout context can support “the connection is suspected under the configured timing assumption” without supporting either “the remote process crashed” or “the remote process did not crash.”

This is not indecision by the mathematician. It is logic that respects incomplete local evidence.

3.10.8 Failure suspicion as local truth

SessionStream’s heartbeat timeout cannot prove remote process death. Network delay, browser pause, or scheduling delay can produce the same silence. The valid proposition is contextual:

$$U_{\text{timing}} \Vdash \text{connection is suspected under policy } P.$$

Strengthening the context with external evidence may refine the proposition. The system should not coerce a local suspicion into a classical global fact.

3.11 A topos-oriented architecture review

For a selected subsystem, ask:

1. What is the base category of contexts?
2. What presheaves carry values, schemas, traces, and laws?
3. Which families are covers?
4. What are the overlaps?
5. Which matching families should glue?
6. Is uniqueness required, existence required, or both?
7. What subobjects express lawful states?
8. What contextual truth values arise when evidence is partial?
9. Which hidden classical assumptions are being made?
10. Which repair changes the base, restriction maps, cover, or sections?

This review is useful even when no formal topos is implemented.

3.12 Chapter laboratory: an executable hydration sheaf test

Build a finite model with cuts 0, 1, 2, 3.

1. Define a deterministic reducer for two event kinds.
2. Enumerate sound snapshots at each cut.
3. Enumerate suffixes with explicit boundary cuts.
4. Define restrictions to the boundary context.
5. Generate matching snapshot/suffix pairs.
6. Reconstruct the full client state.
7. Test existence and uniqueness.
8. Add mutations:
 - drop one suffix event;
 - duplicate an overlap event;
 - reorder two events;
 - change schema version;
 - make the reducer depend on time;
 - overflow and silently drop a buffered batch.
9. Classify each failure using Section 3.7.
10. Decide which mutations produce non-matching families and which produce matching families with no valid amalgamation.

The last distinction is the most important result of the lab.

3.13 Exercises

Exercise 3.1. For a two-open-set cover of an ordinary set, prove that compatible functions into X paste to a unique function.

Exercise 3.2. Give a family of software contexts that covers ordering correctness but does not cover authorization correctness.

Exercise 3.3. Verify the three pretopology axioms for interval covers of a totally ordered event history.

Exercise 3.4. Construct a proposed coverage policy that fails pullback stability when restricted to one session.

Exercise 3.5. For a cover U_1, U_2 of U , write the two maps from $F(U_1) \times F(U_2)$ to $F(U_1 \times_U U_2)$ explicitly.

Exercise 3.6. Prove that the sheaf condition for a two-piece cover is equivalent to the restriction map into the equalizer being a bijection.

Exercise 3.7. Give an example of a separated presheaf that is not a sheaf. Explain existence and uniqueness separately.

Exercise 3.8. Give an example in which multiple local IDs cause two global amalgamations. What extra overlap coordinate restores uniqueness?

Exercise 3.9. Formalize the boundary context B_n for SessionStream hydration. Which fields are essential and which are optional for your chosen client reducer?

Exercise 3.10. Prove Proposition 3.8 by induction on suffix length.

Exercise 3.11. Produce a schedule in which a live UI batch arrives during snapshot load. Trace how hydrating registration and buffering preserve the cover.

Exercise 3.12. Produce a schedule in which a batch arrives after the first buffer drain but before the subscription is marked live. Explain the purpose of the late flush under lock.

Exercise 3.13. Explain why silently dropping a hydration-buffer batch can create a false amalgamation rather than merely delayed consistency.

Exercise 3.14. Classify each of the following: missing EventId, stale schema version, unsorted suffix, inconsistent snapshot cut, context-sensitive redaction, and missing authorization evidence.

Exercise 3.15. State the universal property of sheafification in your own words and draw its factorization triangle.

Exercise 3.16. Suggest a two-step separatedness-then-existence repair for stable retry identity.

Exercise 3.17. Describe sound snapshots as a subobject and state what its characteristic arrow would classify.

Exercise 3.18. For a three-object context poset, list all sieves on the maximal object.

Exercise 3.19. Give a proposition whose truth value is a nontrivial sieve of SessionStream diagnostic refinements.

Exercise 3.20. Explain why a heartbeat timeout supports a local suspicion proposition but not the classical fact of remote crash.

Exercise 3.21. Give a locally witnessed existential proposition in a distributed Session-Stream deployment that lacks a global witness.

Exercise 3.22. Identify one place in ordinary engineering prose where excluded middle is assumed despite partial evidence. Rewrite the claim in contextual form.

Exercise 3.23. Show that every sheaf is separated.

Exercise 3.24. If a presheaf has unique amalgamations but some compatible families have none, which tests would distinguish that from non-matching input?

Exercise 3.25 (chapter synthesis). Specify a finite site for one SessionStream protocol, one presheaf on it, one sheaf law, one contextual predicate, and one mutation that violates each of: restriction functoriality, matching, existence, and uniqueness.

Chapter 4 - What Shape Remains When Local Data Does Not Glue?

The first three chapters gave a diagnostic sequence:

complete coordinates?

-> lawful restriction maps?

-> valid cover?

-> matching local sections?

-> unique global amalgamation?

Only after those questions are answered should we ask whether failure has a higher-dimensional shape.

The geometric intuition enters through the **nerve** of a cover. Vertices represent local contexts. Edges represent genuine pairwise overlaps. Triangles represent genuine three-way overlaps. Tetrahedra represent four-way overlaps, and so on. Missing higher-dimensional simplices can create loops and cavities in the overlap pattern.

Cohomology adds algebra to that shape. It can detect additive discrepancy patterns that remain after all local coordinate changes have been accounted for.

The word “additive” is essential. Arbitrary event payloads, Go structs, and business invariants do not automatically form cochain groups. We will first build the shape, then choose an explicit coefficient system, then compute.

4.1 From an architecture diagram to an overlap complex

4.1.1 Graphs are not yet topology

An ordinary architecture graph might contain arrows such as

EventStore -> Projector -> TimelineStore -> Snapshot -> Client

These arrows describe data flow. A nerve instead describes **joint observability**.

Two contexts get an edge when there is a declared overlap on which their sections can be compared. Three contexts get a filled triangle only when there is a genuine three-way context or witness, not merely because all three pairs have edges.

This distinction is the source of the higher-dimensional intuition.

4.1.2 Simplicial complex

Definition 4.1 (Abstract simplicial complex). An abstract simplicial complex K consists of a set of vertices and a collection of finite nonempty vertex sets, called simplices, such that every nonempty subset of a simplex is also a simplex.

- a one-vertex simplex is a vertex;
- a two-vertex simplex is an edge;
- a three-vertex simplex is a triangle;
- a four-vertex simplex is a tetrahedron.

The dimension of a simplex with $k + 1$ vertices is k .

4.1.3 Nerve of a cover

Definition 4.2 (Nerve). Given a cover $\{U_i \rightarrow U\}$, its nerve is the simplicial complex whose vertices are the cover members and whose k -simplices correspond to nonempty $(k+1)$ -fold overlaps

$$U_{i_0} \times_U \cdots \times_U U_{i_k}.$$

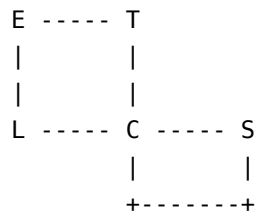
In a general category, “nonempty” must be replaced by whatever criterion says the iterated pullback is a meaningful inhabited context.

4.1.4 SessionStream architecture nerve

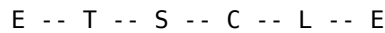
Take five local contexts:

- E : canonical event evidence;
- T : timeline materialization evidence;
- S : snapshot evidence;
- L : live-suffix evidence;
- C : client reconstruction evidence.

Suppose the declared pairwise overlaps are:



A cleaner cycle model is



with edges for event/projector, projector/snapshot, snapshot/client, client/live, and live/event comparisons.

This cycle has a one-dimensional hole **only in the chosen simplicial model** and only if no collection of triangles fills it. If the system has a joint transaction or trace context observing E, T, S together, add the triangle $\{E, T, S\}$. If another context jointly observes

S, C, L , add that triangle. The topology changes because the architecture gained higher-order witnesses.

4.1.5 A face means more than pairwise tests

Suppose these tests exist:

- event cursor agrees with projection cursor;
- projection cursor agrees with snapshot cut;
- event cursor agrees with snapshot cut.

That gives three pairwise edges. A filled triangle means the three comparisons arise inside a joint context and satisfy the three-way relation. Separate tests run at different times do not automatically supply the face.

A database transaction, atomic trace record, or synchronized model state may provide such a face. It does so only when all three values are read or committed as one witness.

4.2 The Cech nerve and repeated overlaps

The abstract nerve records whether overlaps exist. The **Cech nerve** retains the overlaps themselves.

4.2.1 Definition

Definition 4.3 (Cech nerve, working form). For a cover $\{U_i \rightarrow U\}$, the Cech nerve is the simplicial diagram whose degree- k object is the disjoint collection of all $(k + 1)$ -fold overlaps:

$$\check{C}_k = \coprod_{i_0, \dots, i_k} U_{i_0} \times_U \dots \times_U U_{i_k}.$$

Face maps forget one member of the overlap; degeneracy maps repeat one.

For this book, the most important levels are:

- degree 0: local contexts;
- degree 1: pairwise overlaps;
- degree 2: triple overlaps.

4.2.2 SessionStream reading

For a hydration cover:

- degree 0 contains the snapshot and suffix contexts;
- degree 1 contains their boundary overlap;
- higher degrees are repetitions in the two-member cover.

For an architecture cover with event, timeline, snapshot, and client contexts:

- degree 1 contains cursor, identity, and schema overlaps;

- degree 2 contains actual joint witnesses among triples;
- degree 3 contains any four-way audit or transaction context.

The Čech nerve is the bridge from sheaf matching equations to cochain complexes.

4.3 Why cohomology needs coefficients

4.3.1 Additive comparison data

To subtract two local observations or sum discrepancies around a loop, the values must live in an additive structure.

Definition 4.4 (Abelian group). An abelian group is a set A with an addition operation, a zero element, and additive inverses, satisfying associativity and commutativity. The integers $\mathbb{Z}$ under addition are the basic example.

Definition 4.5 (Vector space, working description). A vector space is an abelian group whose elements can also be scaled by numbers from a field such as $\mathbb{R}$; here a field is a number system in which nonzero scalars can be divided. A **linear map** preserves addition and scalar multiplication. Finite tuples of real measurements form vector spaces. The **dimension** is the number of independent coordinates in a basis, where a basis is an independent list from which every vector can be formed uniquely.

Useful SessionStream coefficients include:

- integer cursor offsets;
- parity bits;
- count discrepancies;
- timestamp differences after choosing a unit;
- vector-valued version calibrations;
- linearized conservation laws.

Choosing a **linearization** means selecting numerical coordinates and linear maps that preserve exactly the comparisons being studied. It is a modeling decision, not an automatic translation of arbitrary data.

Less suitable direct coefficients include:

- arbitrary protobuf messages;
- unordered error objects;
- business states with no addition;
- effectful callbacks.

Those can still define set-valued sheaf problems, but not this elementary cochain calculation without further encoding.

4.3.2 Constant coefficients

The simplest coefficient system assigns the same abelian group A to every simplex, with identity restriction maps. This computes ordinary simplicial cohomology with coefficients in A .

For cursor-offset examples, use $A = \mathbb{Z}$ or $\mathbb{R}$.

4.3.3 Sheaf coefficients

A more expressive system assigns different vector spaces to contexts and overlaps, with linear restriction maps. This is a **cellular sheaf** or a linear sheaf on the complex.

We will first compute constant-coefficient cohomology, then generalize.

4.4 Cochains and coboundaries

Choose an **orientation** for every simplex: an ordering of its vertices used to determine signs. Reversing orientation changes the sign of a cochain value but not the resulting cohomology up to canonical isomorphism.

4.4.1 Cochains

Definition 4.6 (k -cochain). For a simplicial complex K and coefficient group A , a k -cochain assigns an element of A to every oriented k -simplex.

The group of k -cochains is written

$$C^k(K; A).$$

For a graph:

- a 0-cochain assigns a value to every vertex;
- a 1-cochain assigns a value to every oriented edge.

4.4.2 Engineering interpretation

A 0-cochain can assign each component its local coordinate origin:

$$x_E, x_T, x_S, x_C.$$

A 1-cochain can assign each interface a measured or declared offset:

$$r_{ET}, r_{TS}, r_{SC}, r_{CE}.$$

4.4.3 Coboundary from vertices to edges

Definition 4.7 (0th coboundary). For a 0-cochain x , its coboundary $\delta^0 x$ is the 1-cochain

$$(\delta^0 x)(v \rightarrow w) = x_w - x_v.$$

It records edge differences induced by vertex coordinates.

If an edge-offset assignment r equals $\delta^0 x$, then the offsets are explained by a choice of local coordinate origins. Such an r is called **exact** or a **coboundary**.

4.4.4 Coboundary from edges to faces

For an oriented triangle $[v_0v_1v_2]$,

$$(\delta^1 r)([v_0v_1v_2]) = r([v_1v_2]) - r([v_0v_2]) + r([v_0v_1]).$$

This is the signed circulation around the triangle boundary.

The **kernel** of an additive map is the set of inputs sent to zero; its **image** is the set of outputs actually attained.

Definition 4.8 (Cocycle). A k -cochain z is a cocycle when

$$\delta^k z = 0.$$

Write

$$Z^k = \ker \delta^k.$$

Definition 4.9 (Coboundary). A k -cochain is a coboundary when it lies in the image of the previous coboundary map:

$$B^k = \text{im } \delta^{k-1}.$$

The identity

$$\delta^{k+1} \circ \delta^k = 0$$

implies every coboundary is a cocycle. A sequence of additive groups and maps with consecutive composites equal to zero is called a **cochain complex**.

4.4.5 Why boundary of boundary is zero

On a triangle, every vertex appears twice with opposite signs when the edge differences are summed. Algebraically, taking differences of differences around a closed face cancels.

This is the basic reason cohomology can quotient cocycles by coboundaries.

4.5 Cohomology

A **quotient** Z^k/B^k treats two elements of Z^k as equivalent when their difference lies in B^k .

Definition 4.10 (Cohomology group). The k th cohomology group is

$$H^k(K; A) = Z^k/B^k = \ker \delta^k / \text{im } \delta^{k-1}.$$

The quotient Z^k/B^k identifies two cocycles when their difference is a coboundary. In other words, it treats discrepancy patterns related by an exact change of local coordinates as the same cohomology class.

4.5.1 Meaning of H^0

A 0-cocycle satisfies $x_w - x_v = 0$ on every edge, so it is constant on each **connected component**, a maximal collection of vertices joined by edge paths. Thus

$$H^0(K; A)$$

records connected-component-wise constant assignments.

For a linear sheaf rather than constant coefficients, H^0 becomes the space of global sections: local values that agree through all restriction maps.

4.5.2 Meaning of H^1

A 1-cocycle is an edge-discrepancy pattern with zero signed sum around every filled triangle. It represents a class in H^1 when it cannot be written as differences of vertex coordinates.

Informally:

H^1 records globally persistent circulation after all local recalibrations have been removed.

This is more precise than “one-dimensional bugs.” The interpretation depends entirely on the chosen complex and coefficients.

4.5.3 Closed versus exact

- **closed:** the discrepancy satisfies all local face constraints;
- **exact:** the discrepancy is explained by assigning a value to each component;
- **cohomology class:** a closed discrepancy modulo exact recalibrations.

A non-closed discrepancy already violates a local higher-order constraint. It is not an H^1 class.

4.6 Worked calculation: a four-component cursor loop

Consider four contexts:

E = event cursor,

T = timeline cursor,

S = snapshot cut,

$C = \text{client cut.}$

Let the edges form a cycle:

$E \dashrightarrow T \dashrightarrow S \dashrightarrow C \dashrightarrow E$

There are no filled triangles in this simplified complex.

4.6.1 Vertex coordinates

A 0-cochain is

$$x = (x_E, x_T, x_S, x_C) \in \mathbb{R}^4.$$

Interpret x_v as the local coordinate origin used by component v .

4.6.2 Edge offsets

A 1-cochain is

$$r = (r_{ET}, r_{TS}, r_{SC}, r_{CE}) \in \mathbb{R}^4.$$

Relative to the ordered vertex and edge bases, the matrix representing δ^0 is the signed **incidence matrix** of the graph. Here it is

$$\delta^0 = \begin{bmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 0 & -1 & 1 \\ 1 & 0 & 0 & -1 \end{bmatrix}.$$

Thus

$$\delta^0 x = (x_T - x_E, x_S - x_T, x_C - x_S, x_E - x_C).$$

Every exact edge assignment has total circulation zero:

$$r_{ET} + r_{TS} + r_{SC} + r_{CE} = 0.$$

Conversely, any edge assignment on this cycle with zero total circulation is exact. Therefore H^1 is one-dimensional over $\mathbb{R}$.

4.6.3 Consistent calibration

Suppose

$$r = (0, 0, 0, 0).$$

All components use the same coordinate convention. The class is zero.

Suppose

$$r = (1, -1, 0, 0).$$

The total circulation is zero. This can be explained by local coordinate choices; for example, shift T by one and return at S . The class is still zero.

Suppose

$$r = (0, 0, 0, 1).$$

The total circulation is one. There is no assignment of vertex coordinates whose differences produce these edge claims. Walking around the architecture returns shifted by one. This is a nonzero cohomology class.

4.6.4 Engineering interpretation

One interface may treat a cursor as “last included ordinal” while another treats it as “next ordinal to consume.” Pairwise adapters can look reasonable, yet composing them around a loop can accumulate an off-by-one.

The cohomology calculation says the mismatch cannot be removed by merely renaming each component’s local origin. At least one transition law or semantic type must change.

4.6.5 What the calculation does not prove

It does not prove a production bug exists. We chose:

- four contexts;
- four edges;
- no faces;
- real-valued additive offsets;
- particular measurements.

A different model may add a face, reveal that an edge is not a genuine overlap, or show the discrepancy is not additive. Cohomology is a diagnostic relative to a formalization.

4.7 Filled triangles and higher-order witnesses

Now add a joint context observing E, T, S together. The complex contains the triangle $[ETS]$.

For a 1-cochain r , the cocycle condition on that face is

$$r_{ET} + r_{TS} - r_{ES} = 0.$$

If the pairwise edge claims violate this equation, the discrepancy is not closed. The triple witness catches it locally.

This illustrates two distinct roles of a higher-dimensional cell:

1. it adds a local consistency equation;
2. it can fill a topological loop, reducing H^1 .

4.7.1 Transaction as a face

Suppose event append, timeline apply, and projection cursor advance are committed in one database transaction. A trace of that transaction can form a three-way witness among event, materialization, and cursor contexts.

Representing it as a face is justified only when the transaction's state maps to all three edge comparisons and makes the boundary commute.

4.7.2 Pairwise atomicity is insufficient

Three pairwise transactions do not necessarily create one triple face. The reads or commits may occur at different moments. Higher-order compatibility requires a joint witness, not just all pairwise witnesses.

4.8 Cellular sheaves: different data at different cells

Constant coefficients assume every context carries the same kind of value. SessionStream contexts usually carry different spaces.

4.8.1 A simple linear sheaf model

Assign:

- a vector space $F(v)$ to every component vertex;
- a vector space $F(e)$ to every overlap edge;
- a linear restriction map from each incident vertex space into the edge space.

For an oriented edge $e = (v, w)$, write

$$\rho_{v,e} : F(v) \rightarrow F(e), \quad \rho_{w,e} : F(w) \rightarrow F(e).$$

A 0-cochain is a tuple of local values $x_v \in F(v)$. Define

$$(\delta^0 x)_e = \rho_{w,e}(x_w) - \rho_{v,e}(x_v).$$

Then

$$H^0 = \ker \delta^0$$

is exactly the space of global sections: assignments whose overlap observations agree.

4.8.2 SessionStream example

Let:

- $F(E)$ contain (session, eventOrdinal, eventId) vectors after linear encoding;
- $F(T)$ contain (session, representedOrdinal, entityDigest);
- $F(S)$ contain (session, snapshotOrdinal, snapshotDigest);
- edge spaces contain the shared coordinates each pair can compare.

Restriction maps extract or linearly transform shared coordinates. A global section is a family of component values agreeing on all overlaps.

4.8.3 Interpreting H^1

For a cellular sheaf, H^1 can measure independent overlap discrepancy classes not resolved by local vertex assignments. Depending on the sheaf, it may indicate obstructions, ambiguity, or degrees of freedom.

One must inspect the exact cochain complex before attaching an engineering meaning.

4.9 An obstruction catalogue for SessionStream

4.9.1 Cursor holonomy: a good cohomology candidate

Holonomy is the net transformation accumulated by transporting a coordinate around a closed loop. In the additive cursor model, it is simply the signed sum of edge offsets around that loop.

Symptom. Cursor translations compose around a loop to a nonzero shift.

Model. Integer or real edge offsets on an architecture nerve.

Likely tool. H^1 or direct cycle circulation.

Repair. Separate semantic cursor types and correct transition maps; add joint calibration witnesses.

4.9.2 Snapshot race: usually a missing joint witness first

Symptom. Snapshot cut and entity rows come from different database states.

Model. Two local observations with no common transaction apex.

Likely tool. Pullback/limit failure or non-matching sections, before cohomology.

Repair. Read transaction or versioned snapshot.

4.9.3 Stable retry identity: usually a missing coordinate

Symptom. Redelivery receives a fresh ordinal and appears as a new event.

Model. Restriction forgets logical event identity.

Likely tool. Fiber analysis and base-category repair.

Repair. Stable `EventId`, producer identity, or stable stream coordinate.

4.9.4 Deterministic replay: usually functoriality

Symptom. Rebuild differs from live processing for the same event prefix.

Model. Replay is not a well-defined functor on declared inputs.

Likely tool. Naturality/functor tests and hidden-coordinate search.

Repair. Declare all inputs or remove hidden effects.

4.9.5 Hydration gap: usually cover failure

Symptom. An accepted event is neither represented in the snapshot nor delivered in the suffix.

Model. Snapshot and retained suffix do not cover the execution.

Likely tool. Sheaf coverage and existence test.

Repair. Earlier hydrating registration, sufficient buffer, explicit overflow/reconnect.

4.9.6 Conflicting migrations: possible cocycle problem

Symptom. Pairwise schema translations work, but translating around a version cycle returns changed data.

Model. Transition functions on edges of a version cover.

Likely tool. A coherence calculation using composed migration functions. Ordinary additive H^1 applies only after a justified linear or abelian encoding; the more general group-valued theory is beyond this book.

Repair. Canonical version, coherent migration composition, or elimination of cyclic paths.

4.9.7 Observer evidence loss: epistemic limitation

Symptom. Best-effort observer records are dropped under backpressure.

Model. The observer presheaf does not cover the authoritative execution trace.

Likely tool. Coverage and local-truth analysis.

Repair. Weaken claims made from observer data, add durable flight recording, or strengthen the observation contract.

4.10 Computing cohomology from matrices

For a finite simplicial complex over a field, coboundary maps are matrices.

4.10.1 Workflow

1. Order the vertices, edges, and faces.
2. Choose orientations.
3. Build $\delta^0 : C^0 \rightarrow C^1$.
4. Build $\delta^1 : C^1 \rightarrow C^2$.
5. Verify $\delta^1 \delta^0 = 0$.
6. Compute ranks and nullities. The **rank** is the dimension of a linear map's image; the **nullity** is the dimension of its kernel.
7. Use

$$\dim H^k = \dim \ker \delta^k - \dim \operatorname{im} \delta^{k-1}.$$

8. Interpret basis representatives only after returning to the software model.

4.10.2 Graph cycle rank

For a connected graph with V vertices and E edges and no 2-simplices,

$$\dim H^1 = E - V + 1$$

over a field.

For a graph with c connected components,

$$\dim H^1 = E - V + c.$$

Adding filled triangles can reduce this dimension because their boundaries impose equations and become boundaries.

4.10.3 Numerical caution

Over $\mathbb{R}$, numerical rank requires a tolerance. Integer coefficients can carry additional finite-order information, but that computation is beyond this introductory route. Real linear algebra is sufficient for the first SessionStream diagnostics in this chapter.

4.11 Build ss-sheafcheck: an executable research tool

The purpose of a tool is not to announce “cohomology found a bug.” It is to force the model to be explicit and to preserve the diagnostic order from earlier chapters.

4.11.1 Proposed package layout

cmd/ss-sheafcheck/	
internal/model/	contexts, arrows, covers, cells
internal/presheaf/	section sets and restriction maps
internal/matching/	overlap compatibility
internal/gluing/	global-section search

internal/nerve/	simplicial-complex construction
internal/cochain/	matrices and coefficient systems
internal/trace/	SessionStream trace adapters
internal/report/	counterexamples and explanations
examples/	

4.11.2 Model file

A model should state contexts and maps rather than infer them from names:

```
contexts:
  event:
    fields: [session, event_ordinal, event_id]
  timeline:
    fields: [session, represented_ordinal, entity_digest]
  snapshot:
    fields: [session, snapshot_ordinal, entity_digest]

overlaps:
- name: event_timeline
  left: event
  right: timeline
  compare: ordinal
- name: timeline_snapshot
  left: timeline
  right: snapshot
  compare: [ordinal, entity_digest]

covers:
- whole: reconstructed_session
  members: [event, timeline, snapshot]

faces:
- [event, timeline, snapshot]
```

The schema should distinguish:

- an overlap edge;
- a cover declaration;
- a higher-order face;
- an additive coefficient map.

4.11.3 Validation phases

Phase 1: base validation. Check category identities, compositions, and pullback references.

Phase 2: presheaf validation. Check restriction identity and composition laws.

Phase 3: cover validation. Check identity, composition, and pullback stability where finite enumeration is possible.

Phase 4: matching. For supplied local sections, compute overlap restrictions and report mismatches.

Phase 5: gluing. Search for global sections; distinguish zero, one, and multiple amalgamations.

Phase 6: nerve. Build vertices, edges, and higher simplices from declared overlaps and faces.

Phase 7: cohomology. Only for declared additive coefficient systems, construct coboundary matrices and report classes.

4.11.4 Diagnostic output

A useful report should say:

```
FAILURE CLASS: non-functorial restriction
CONTEXT PATHS:
  snapshot_v3 -> snapshot_v2 -> snapshot_v1
  snapshot_v3 -> snapshot_v1
COUNTEREXAMPLE SECTION:
  session = s-7
  entity = assistant-message-3
DIFFERENCE:
  status field: "finished" vs "streaming"
  or
FAILURE CLASS: nonzero cursor circulation
CYCLE:
  event -> timeline -> snapshot -> client -> event
EDGE OFFSETS:
  [0, 0, 0, 1]
CIRCULATION:
  1
LOCAL RECALIBRATION POSSIBLE:
  no
```

The report must retain enough source evidence to reproduce the result.

4.11.5 Trace adapter

SessionStream already records typed observer and refinement traces. A trace adapter can map records to local sections:

- SnapshotLoaded supplies snapshot cut and entity summary;
- SnapshotSent supplies transport visibility;
- UIEventBuffered supplies suffix evidence;
- HydrationBufferFlushed supplies ordered flush evidence;
- SubscriptionLive supplies transition state;

- drop counters limit which global claims are justified.

A trace is evidence of one execution, not proof over all executions. Property testing, model checking, or theorem proving can explore broader spaces.

4.11.6 Capstone research questions

1. Can a finite site capture snapshot-before-live precisely enough that every protocol mutation produces a classified counterexample?
2. Can cursor semantics be extracted from Go types and adapter code into an architecture cochain automatically?
3. Can schema migrations be checked as natural transformations over generated event prefixes?
4. Can observer loss be represented by a contextual truth operator describing which authoritative propositions remain knowable?
5. Can a transaction boundary be inferred as a proposed higher-dimensional face and validated against traces?

4.12 A final diagnostic order

When a shape looks like a hole, proceed in this order:

1. **Vocabulary:** Are event identity, ordinal, cursor, cut, and version separate concepts?
2. **Coordinates:** Are all relevant inputs represented?
3. **Category:** Do transformations compose and identities behave correctly?
4. **Presheaf:** Are restrictions genuine and functorial?
5. **Site:** Do declared covers remain covers under refinement?
6. **Matching:** Do local observations agree on explicit overlaps?
7. **Gluing:** Does a unique global section exist?
8. **Nerve:** What higher-order overlaps are present or absent?
9. **Coefficients:** Is the discrepancy additive in a declared group or vector space?
10. **Cohomology:** Is there a closed non-exact discrepancy class?

This order prevents a sophisticated invariant from obscuring a simpler modeling error.

4.13 Exercises

Exercise 4.1. Build the nerve of a three-member cover in which every pair overlaps but the triple overlap is empty. Draw the boundary triangle without a filled face.

Exercise 4.2. Give a SessionStream interpretation of Exercise 4.1 in which three pairwise tests exist but no joint transaction or trace context exists.

Exercise 4.3. Add a genuine triple witness to Exercise 4.2. Explain why this justifies filling the triangle.

Exercise 4.4. Distinguish an architecture data-flow edge from a nerve overlap edge using two concrete examples.

Exercise 4.5. Write degrees 0, 1, and 2 of the Čech nerve for a cover by three contexts.

Exercise 4.6. Choose an arbitrary payload discrepancy and explain why it does not yet define a 1-cochain.

Exercise 4.7. Define an integer-valued coefficient system for cursor offsets on four SessionStream components.

Exercise 4.8. For an oriented triangle, compute $\delta^1 \delta^0 x$ directly and show it is zero.

Exercise 4.9. Prove that exact 1-cochains on the four-cycle have zero total circulation.

Exercise 4.10. Prove the converse: every real 1-cochain on the four-cycle with zero circulation is exact.

Exercise 4.11. Compute a basis for H^1 of the four-cycle over $\mathbb{R}$.

Exercise 4.12. Interpret the cochain $(1, -1, 0, 0)$ as local cursor conventions and construct vertex values whose coboundary it is.

Exercise 4.13. Show that $(0, 0, 0, 1)$ is not exact on the four-cycle.

Exercise 4.14. Fill one triangle in a square divided by a diagonal. Build δ^1 and compute $\dim H^1$.

Exercise 4.15. Explain why a nonzero triangle circulation in a filled complex is not an H^1 class but a failure to be a cocycle.

Exercise 4.16. Define a simple cellular sheaf with different vertex and edge spaces for event, timeline, and snapshot ordinals. Compute its H^0 .

Exercise 4.17. Give an example where H^0 contains multiple global sections. Does that mean the sheaf condition fails? Explain the difference between multiple global states and multiple amalgamations of one matching family.

Exercise 4.18. Classify each issue in Section 4.9 by the earliest chapter-level concept that detects it.

Exercise 4.19. Model a schema-migration cycle. Which parts can be encoded as additive offsets, and which require reasoning directly about composed migration functions?

Exercise 4.20. Design a mutation of cursor translation that creates nonzero circulation while all individual edge unit tests still pass.

Exercise 4.21. Write pseudocode that constructs an incidence matrix from an ordered list of vertices and oriented edges.

Exercise 4.22. Write pseudocode that verifies $\delta^1 \delta^0 = 0$ before computing cohomology.

Exercise 4.23. Specify the minimum trace fields needed to derive the hydration cover and matching conditions from one WebSocket run.

Exercise 4.24. Explain which claims become unjustified when observer records can be dropped.

Exercise 4.25. Extend the `ss-sheafcheck` model schema with an explicit distinction between authoritative sections and diagnostic observations.

Exercise 4.26. Design a report for a matching family with no amalgamation and a separate report for a family with two amalgamations.

Exercise 4.27. Construct a finite example in which adding a missing coordinate removes an apparent cohomology class because the original edges were incorrectly defined.

Exercise 4.28. Construct a finite example in which adding a higher-dimensional face removes an actual H^1 generator.

Exercise 4.29. Explain why zero H^1 does not prove all SessionStream invariants.

Exercise 4.30 (capstone). Implement or fully specify one end-to-end study:

- extract local sections from a SessionStream trace;
- validate restriction laws;
- check a cover and matching family;
- search for global reconstruction;
- build the nerve;
- define one additive coefficient system;
- compute H^0 and H^1 ;
- explain the result without using the phrase “topological bug” unless the model supports it.

Closing Perspective

The productive claim is not that software systems secretly are topological spaces. It is that software engineering repeatedly deals with context-indexed partial information and makes global claims from it.

Category theory asks whether transformations compose.

Limits ask which observations have a universal joint witness.

Presheaves organize what can be known at each context and how richer knowledge is forgotten.

Sheaves state when compatible local knowledge determines a unique whole.

Topos theory provides a universe in which context-dependent values have functions, predicates, and an internal logic.

Nerves turn overlap patterns into multidimensional shapes.

Cohomology detects selected additive discrepancy patterns that remain globally twisted after local coordinate changes.

For SessionStream, the research route is therefore:

runtime traces

- > explicit contexts and sections

- > restriction laws

- > covers and matching

- > global executions

- > nerves and obstruction diagnostics

The next practical step is not a larger metaphor. It is one small site, one explicit presheaf, one executable sheaf condition, and one coefficient system whose engineering meaning is defensible.

Appendix A - Definition and Dependency Map

This appendix is a review tool, not a substitute for the motivating examples in the chapters. Read each entry in both directions:

- from the term to the SessionStream question it answers;
- from an engineering failure back to the earliest mathematical term that explains it.

A.1 Arrows and universal constructions

Term	Working meaning	SessionStream anchor
category	objects and composable arrows with identities and associative composition	prefixes and extensions; representations and codecs
domain / codomain	source and target of an arrow	input and output context of a transformation
commutative diagram	all directed paths with common endpoints have equal composites	migrate-then-replay equals replay-then-migrate
isomorphism	reversible arrow preserving the chosen structure	two snapshot representations with identical observables
monic	left-cancellable arrow	inclusion of lawful states into all typed states
epic	right-cancellable arrow	a coverage-like surjection in Set, but not generally identical to surjectivity
universal property	every candidate factors uniquely through the universal candidate	representation-independent contract
product	universal independent pairing	observe snapshot metadata and entity data together
equalizer	universal subobject on which two arrows agree	live replay equals rebuild; no snapshot violations

Term	Working meaning	SessionStream anchor
diagram	a schema of objects and required arrows	event, cursor, timeline, snapshot relationship
cone	one candidate joint witness over a diagram	one execution inducing all component observations
limit	universal compatible cone	canonical coherent state compatible with every local view
terminal object	unique target from every object	final cut in a bounded prefix category
pullback	universal pair agreeing over shared meaning	typed join of cursor claims; authorized request
finite completeness	existence of all finite limits	ability to form finite coherent predicates and joins
exponential	internal function object with evaluation and currying	fully declared projection behavior
Cartesian closed	finite products plus exponentials	behavior can be represented inside the category

A.2 Functors and presheaves

Term	Working meaning	SessionStream anchor
functor	preserves objects, arrows, identities, and composition	replay interpretation; codec interpretation
faithful	does not identify distinct source arrows	encoding retains distinctions among event transformations
full	every target arrow between images comes from the source	target behavior does not invent unsupported transformations
natural transformation	coherent family of maps between functors	migration compatible with every prefix extension
opposite category	same objects with arrows reversed	context inclusion becomes data restriction
contravariant functor	functor from the opposite category	richer observation restricts to poorer observation
context	declared boundary of available information	session/cut/aspect tuple; API parameter subset

Term	Working meaning	SessionStream anchor
presheaf	context-indexed sets with functorial restrictions	traces, states, schemas, local observations
section	one local value over one context	one trace or snapshot observation
global section	compatible value across the whole context system	one coherent execution assignment
fiber	all full states with the same exposed parameters	possible semantic completions of an API request
representable presheaf	probes into one object	observable tests of a snapshot implementation
Yoneda lemma	an element is determined by its coherent behavior under probes	interface-oriented equivalence of implementations

A.3 Sheaves, sites, and truth

Term	Working meaning	SessionStream anchor
cover	declared family jointly sufficient for a whole context	snapshot plus boundary-aware live suffix
pretopology	covers stable under identity, composition, and pullback	local coverage remains valid under session/time refinement
site	category plus a local-coverage policy	SessionStream observation contexts with temporal/aspect covers
overlap	pullback of two cover members over the whole	shared cut, identity, schema, or digest evidence
matching family	local sections equal after restriction to every overlap	snapshot and suffix agree at their boundary
amalgamation	global section restricting to all local sections	reconstructed client execution
separated presheaf	matching data has at most one amalgamation	identity and order are unambiguous when reconstruction exists
sheaf	every matching family has exactly one amalgamation	sound, complete, deterministic snapshot-plus-suffix protocol

Term	Working meaning	SessionStream anchor
sheafification	universal map from a presheaf into a sheaf	canonicalization plus completion, used only as an analogy unless formalized
subobject	lawfully included part of an object	sound snapshots among all typed snapshots
subobject classifier	universal truth-value object classifying subobjects	characteristic arrow for snapshot soundness
sieve	refinement-closed family of probes into a context	all evidence-bearing refinements that verify a law
Grothendieck topology	axioms selecting covering sieves	sieve-based local coverage
topos	finite limits, exponentials, and a subobject classifier	universe of context-dependent sets, functions, and predicates
local truth	truth evaluated at a context and stable under refinement	timeout implies suspicion under a timing policy, not remote death
intuitionistic logic	evidence-sensitive logic without automatic excluded middle	partial operational evidence may prove neither a claim nor its negation

A.4 Nerves and cohomology

Term	Working meaning	SessionStream anchor
simplicial complex	vertices, edges, faces, and higher simplices closed under taking subsets	contexts and their higher-order overlaps
nerve	simplex for every inhabited iterated overlap of cover members	architecture shape of joint observability
Cech nerve	diagram retaining the actual repeated overlaps	pairwise and triple comparison contexts
abelian group	additive values with zero and inverses, addition commutative	integer cursor offsets
vector space	additive values also scalable by field elements	real-valued measurement vectors
cochain	value assigned to every oriented simplex of one dimension	local coordinates on vertices or offsets on edges

Term	Working meaning	SessionStream anchor
coboundary map	turns local values into signed boundary differences	component coordinates induce interface offsets
cocycle	cochain whose next coboundary vanishes	edge discrepancy satisfies all filled-face equations
coboundary	discrepancy induced by lower-dimensional local choices	offset removable by recalibrating components
cohomology	cocycles modulo coboundaries	persistent circulation after local recalibration
holonomy	transformation accumulated around a closed loop	off-by-one cursor shift after traversing adapters
cellular sheaf	different linear spaces on cells with restriction maps	event, timeline, and snapshot comparison spaces

A.5 Failure-to-concept index

Symptom	First concept to inspect
projector output changes on replay	functoriality and missing coordinates
direct restriction differs from staged restriction	presheaf composition law
snapshot and suffix use different schema versions	matching-family failure
event omitted from both snapshot and suffix	bad cover or no amalgamation
same local data reconstructs two identity assignments	separatedness failure
entity rows exceed the advertised cut	missing limit/pullback witness
cursor adapters accumulate an off-by-one around a loop	1-cochain circulation and possibly H^1
observer says nothing about an event because its record was dropped	epistemic coverage limitation
parameter set cannot decide an invariant	invariant not constant on fibers
pairwise tests pass but triple transaction law fails	missing face or non-cocycle on a filled face

Appendix B - Selected Hints and Solutions

These sketches show the expected style of reasoning. They deliberately leave routine algebra and implementation details for the reader.

B.1 Chapter 1

Exercise 1.6: uniqueness of products

Let (P, p_A, p_B) and (Q, q_A, q_B) be products. Universality of P applied to the pair (q_A, q_B) gives a unique $u : Q \rightarrow P$. Universality of Q applied to (p_A, p_B) gives a unique $v : P \rightarrow Q$.

Both $u \circ v$ and 1_P have projection composites p_A and p_B , so product uniqueness gives $u \circ v = 1_P$. Similarly $v \circ u = 1_Q$. If u' is another projection-preserving map $Q \rightarrow P$, product uniqueness gives $u' = u$. Thus the isomorphism is unique under the structure-preserving condition.

Exercise 1.10: pullback in sets

Let

$$P = \{(a, b) \in A \times B \mid f(a) = g(b)\}$$

with projections p_A, p_B . The square commutes by membership in P . Given $x_A : X \rightarrow A$ and $x_B : X \rightarrow B$ with $f x_A = g x_B$, define

$$u(x) = (x_A(x), x_B(x)).$$

The compatibility equation ensures $u(x) \in P$. Projection equations hold by definition. Any other map with those projections must have exactly the same ordered pair at every x , so it equals u .

Exercise 1.14: constant on fibers

Assume I is constant on fibers of $r : X \rightarrow P$. For p in the image of r , choose any x with $r(x) = p$ and define $\bar{I}(p) = I(x)$. Fiber constancy makes this independent of the choice. Then $I = \bar{I} \circ r$. Uniqueness follows because every p in the image has some preimage.

Conversely, if $I = \bar{I} \circ r$ and $r(x) = r(y)$, then

$$I(x) = \bar{I}(r(x)) = \bar{I}(r(y)) = I(y).$$

Exercise 1.19: pullback from product and equalizer

On $A \times B$, compare

$$f \circ \pi_A \quad \text{and} \quad g \circ \pi_B.$$

Their equalizer includes exactly those pairs whose images in C agree. Compose the equalizer inclusion with the product projections. The product and equalizer universal properties combine to give the pullback universal property.

B.2 Chapter 2

Exercise 2.3: hidden randomness

The same source object and arrow must have one target object and arrow under a functor. If replay of one fixed prefix can produce either state 7 or state 8 because of hidden randomness, the proposed object assignment is not a function. Add the random seed to the source context, record the random outcome as an event, or remove randomness.

Exercise 2.6: pointwise migration without naturality

Let old replay append text exactly, while new replay uppercases each new chunk. Define the migration at each cut to uppercase the whole accumulated text **except** at cut 1, where it leaves the text unchanged. Every component is a function, but the square for the extension from cut 1 to cut 2 fails. The example shows that a family of conversions is weaker than a natural transformation.

Exercise 2.11: composition-law mutation

Let prefix restriction normally truncate to the requested ordinal. Mutate the direct restriction from 3 to 1 so that it retains an event at ordinal 2, while restrictions 3 to 2 and 2 to 1 remain correct. Identity tests pass, but

$$\rho_1^3 \neq \rho_1^2 \circ \rho_2^3.$$

A property test must sample composable triples, not only identities.

Exercise 2.17: representable on a three-object chain

Let $0 \rightarrow 1 \rightarrow 2$ and take $A = 2$. Then

$$y_2(0) = \{0 \rightarrow 2\},$$

$$y_2(1) = \{1 \rightarrow 2\},$$

$$y_2(2) = \{1_2\}.$$

Each is a singleton in a poset category. If instead $A = 1$, then $y_1(2)$ is empty because no arrow $2 \rightarrow 1$ exists, while $y_1(0)$ and $y_1(1)$ are singletons. Restrictions are precomposition.

B.3 Chapter 3**Exercise 3.6: equalizer form**

The restriction map

$$r : F(U) \rightarrow F(U_1) \times F(U_2)$$

lands in the equalizer because both restrictions of one global section to the overlap are equal by presheaf composition. If F is a sheaf, every compatible pair has a unique amalgamation, so r is bijective onto the equalizer. Conversely, such a bijection provides exactly one amalgamation for every matching pair.

Exercise 3.7: separated but not a sheaf

On a disconnected region, let $F(U)$ be constant functions $U \rightarrow \mathbb{R}$. Restrictions remain constant. Two local constant functions on disjoint cover members are automatically compatible because the overlap is empty, but they glue only when they use the same constant. If a gluing exists it is unique, so the presheaf is separated but fails existence.

Exercise 3.10: hydration induction

Base case: an empty suffix has the unique amalgamation S_n . Inductive step: assume the state through k is uniquely determined. The next ordered event has one deterministic reducer result, so the state through $k + 1$ is uniquely determined. Any other amalgamation must agree through k by induction and then agree after the same final reducer step.

Exercise 3.12: late buffer

After the first drain, a new batch can arrive while the subscription is still hydrating. If the state were marked live before that late batch was queued, a newer live batch could overtake

it. Holding the subscription lock while flushing late batches and switching to live gives a linearization point: all hydrating batches are queued before any producer can observe the live state.

Exercise 3.18: sieves on a three-object chain

For $0 \rightarrow 1 \rightarrow 2$, arrows into 2 are 1_2 , $1 \rightarrow 2$, and $0 \rightarrow 2$. A sieve must be closed under precomposition. The sieves are:

- the empty sieve;
- $\{0 \rightarrow 2\}$;
- $\{1 \rightarrow 2, 0 \rightarrow 2\}$;
- the maximal sieve containing all three arrows.

A set containing $1 \rightarrow 2$ but not $0 \rightarrow 2$ is not a sieve because precomposition by $0 \rightarrow 1$ is missing.

B.4 Chapter 4

Exercise 4.8: boundary of boundary

For vertex values x_0, x_1, x_2 ,

$$(\delta^0 x)_{01} = x_1 - x_0,$$

$$(\delta^0 x)_{12} = x_2 - x_1,$$

$$(\delta^0 x)_{02} = x_2 - x_0.$$

Therefore

$$\delta^1 \delta^0 x = (x_2 - x_1) - (x_2 - x_0) + (x_1 - x_0) = 0.$$

Exercise 4.10: zero circulation implies exactness

Set $x_E = 0$. Define

$$x_T = r_{ET},$$

$$x_S = r_{ET} + r_{TS},$$

$$x_C = r_{ET} + r_{TS} + r_{SC}.$$

The first three edge equations hold. The final equation requires

$$x_E - x_C = r_{CE},$$

which is exactly the zero-circulation equation. Hence the cochain is a coboundary.

Exercise 4.14: square with a diagonal and one filled triangle

Take vertices 0, 1, 2, 3, cycle edges 01, 12, 23, 30, diagonal 02, and fill triangle 012. The graph alone has cycle rank $5 - 4 + 1 = 2$. The filled face kills the cycle $01 + 12 - 02$, leaving one independent class represented by the outer route $02 + 23 + 30$. Thus $\dim H^1 = 1$ over a field.

Exercise 4.17: multiple global sections

A sheaf may have many global sections. The sheaf condition says that **one fixed matching family** has exactly one amalgamation. Different matching families can have different amalgamations. For a constant real-valued sheaf on a connected region, every real constant gives a different global section, while each compatible family determines one of them uniquely.

Exercise 4.29: zero H^1 is limited evidence

Zero H^1 only says that every closed 1-cochain in the chosen additive model is exact. It says nothing about:

- omitted coordinates;
- non-functorial restrictions;
- invalid covers;
- non-matching sections;
- nonlinear or nonadditive invariants;
- higher-degree obstructions;
- implementation behaviors absent from the model.

Appendix C - Source Map and Parallel Study Route

C.1 Goldblatt route

The custom sequence corresponds to the supplied book as follows.

This book	Goldblatt material used for the route
Chapter 1	Chapter 3, especially products, equalizers, limits, pullbacks, completeness, and exponentiation
Chapter 2	Chapter 9 on functors and natural transformations; opening of Chapter 14 on presheaves/stacks
Chapter 3	Chapter 4 on subobjects, classifiers, topoi, bundles and sheaves; Chapter 14 on sheaves, sites, and local truth
Chapter 4	applied extension beyond Goldblatt's main route, using the sheaf concepts already developed

Goldblatt often uses the term **stack** for what this book calls a presheaf in the Chapter 14 discussion. Modern usage varies, and “stack” now often means a categorified sheaf-like object, so this book consistently uses **presheaf** for set-valued contravariant functors.

C.2 SessionStream source map

The frozen study uses these core paths:

Concern	Source path
commands, events, sessions	pkg/sessionstream/types.go
UI and timeline projections	pkg/sessionstream/projection.go
hub processing and replay	pkg/sessionstream/hub.go
event, hydration, and cursor interfaces	pkg/sessionstream/hydration.go

Concern	Source path
ordinal assignment	pkg/sessionstream/ordinals.go
schema registry	pkg/sessionstream/schema.go
SQLite materialization and snapshot	pkg/sessionstream/hydration/sqlite/store.go
snapshot-before-live adapter	pkg/sessionstream/transport/ws/server.go
transport observer traces	pkg/sessionstream/transport/ws/observer*.go
typed chat projection example	examples/chatdemo/chat.go

The Architecture Garden study emphasizes six open or central law families used throughout this book:

1. per-session serializability;
2. consistent-cut snapshots;
3. stable retry identity;
4. atomic projection progress;
5. deterministic replay;
6. snapshot-plus-suffix completeness.

C.3 Twelve-week route

Week	Reading and work
1	Chapter 1 through products and equalizers; redraw every universal diagram
2	Limits and pullbacks; specify the SQLite snapshot witness
3	Exponentials and declared projector dependencies; complete Chapter 1 exercises
4	Functors and natural transformations; write replay and migration laws
5	Opposite categories and context design; define one two-dimensional context base
6	Presheaves and fibers; implement restriction property tests
7	Covers, pretopologies, matching families; classify four failure traces
8	Hydration gluing theorem; implement the finite sheaf lab

	Week	Reading and work
	9	Toposes, subobjects, sieves, and local truth; rewrite one operational claim intuitionistically
	10	Build a nerve from declared SessionStream overlaps
	11	Compute the four-cycle H^1 example and one cellular H^0 example
	12	Specify or implement an ss-sheafcheck slice and write a research report

C.4 Study notebook template

For each reading session, record:

Term:

Why it was needed:

Set example:

Formal definition:

SessionStream objects:

SessionStream arrows/restrictions:

One lawful example:

One counterexample:

One exercise attempted:

One implementation or test implication:

Remaining confusion:

The “remaining confusion” line is mandatory. It prevents fluent vocabulary from being mistaken for understanding.

C.5 Completion criteria

You have completed the book’s core route when you can do all of the following without consulting the glossary:

- define a limit by its cone universal property;
- distinguish product, equalizer, and pullback in one API example;
- define a category of SessionStream contexts;
- give a presheaf and prove its restriction laws;
- distinguish a cover from an arbitrary list of components;
- compute overlap restrictions and identify a matching family;
- separate existence from uniqueness in the sheaf condition;
- state the hydration gluing theorem and name each implementation premise;
- explain a sieve as a contextual truth value;

- build a nerve with a justified face;
- define cochains, cocycles, coboundaries, and H^1 ;
- explain why a nonzero circulation is meaningful only relative to explicit coefficients;
- classify a bug before reaching for cohomology.